

BCA OS Exam Handbook

Complete Exam Handbook: A Professional, Exam-Oriented Reference for BCA Students

Covers: Unit I – VI (Full Syllabus) | Definitions | Diagrams | Numericals | Viva Questions | MCQs | PYQs | Revision Notes

Table of Contents

Unit	Topic	Page (approx.)
I	Introduction to OS, Types of OS, Process Concepts	3
II	CPU Scheduling & Deadlock	—
III	Memory Management	—
IV	File System	—
V	Disk Management	—
VI	Protection and Security	—
—	Master Question Bank (Viva / 1M / 2M / 5M / Long)	—
—	Formula Sheet & Last Night Revision Notes	—

How to Use This Handbook

1. Read the theory first, then study the solved numericals.
2. Every algorithm contains: Theory → Working → Pseudocode → Solved Example → Advantages → Disadvantages → Applications.
3. Practice the Gantt charts and numericals by hand. Most OS exams give 40% to 50% weight to

numerical-based questions (CPU Scheduling, Deadlock/Banker's, Page Replacement, Disk Scheduling).

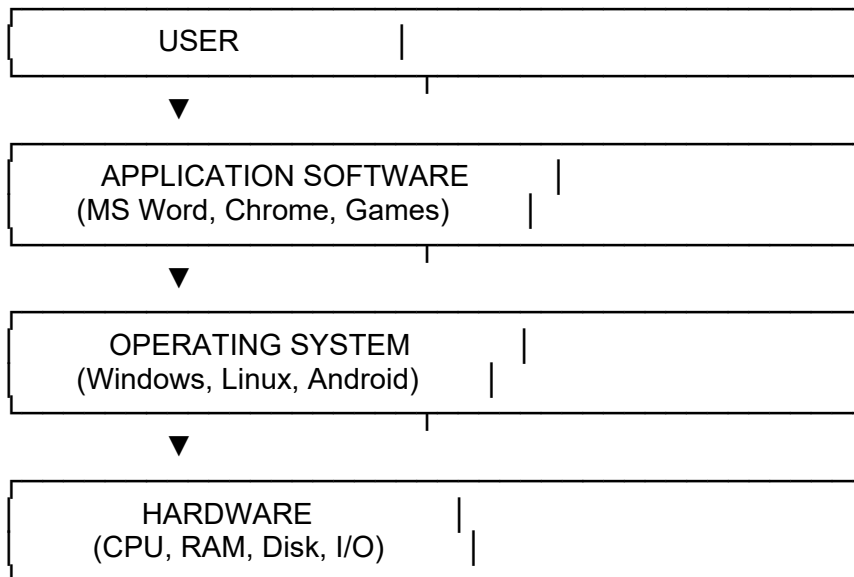
4. Use the **Memory Trick** boxes for quick recall during exams.
5. Revise the **Last Night Revision Notes** section one day before the exam.

UNIT I — OPERATING SYSTEM: INTRODUCTION & PROCESS CONCEPTS

1.1 What is an Operating System?

- **Definition:** An Operating System (OS) is system software that acts as an interface between the user and computer hardware. It manages hardware resources (CPU, memory, I/O devices) and provides services to application programs.
- **Simple Analogy:** Think of a computer as a large office building. The hardware (CPU, RAM, disk, keyboard) represents the rooms, furniture, and equipment. The user is a visitor who wants work completed. The Operating System acts as the receptionist or manager who understands what the visitor needs and instructs the staff (hardware) to execute it. Without this manager, the visitor cannot interact directly with the building infrastructure.

System Architecture Layout



- **Examples of OS:** Windows, Linux, macOS, Android, iOS, Unix.

1.2 Objectives of an Operating System

- **Convenience:** Makes the computer system easy and convenient to use.
- **Efficiency:** Ensures efficient utilization of computer hardware resources.
- **Ability to Evolve:** The OS should be built in a manner that allows new features to be added without disrupting existing services.
- **Throughput:** Maximizes the amount of work completed within a given timeframe.

💡 **Memory Trick:** C-E-A-T → Convenience, Efficiency, Ability to Evolve, Throughput.

1.3 Functions of an Operating System

Function	Description
Process Management	Creating, scheduling, and terminating processes.
Memory Management	Allocating and deallocating memory space.
File Management	Creating, deleting, and organizing files and directories.
Device Management	Controlling I/O devices via device drivers.
Security & Protection	Protecting data and system resources from unauthorized access.
User Interface	Providing a Command Line Interface (CLI) or Graphical User Interface (GUI).
Error Detection	Detecting and handling hardware or software errors.
Resource Allocation	Allocating resources (CPU, memory) to multiple users or programs fairly.

💡 **Memory Trick:** "Please Manage File Devices Securely Under Everyone's Rules" → Process, Memory, File, Device, Security, UI, Error, Resource.

1.4 Types of Operating Systems

1.4.1 Batch Operating System

- Jobs with similar processing requirements are grouped (batched) and executed together without direct user interaction during execution.
- **Example:** Payroll systems, early mainframe systems.
- **Advantage:** No manual intervention is needed once the batch execution starts; highly efficient for large, repetitive jobs.

- **Disadvantage:** Difficult to debug code; the CPU may sit idle while waiting for slow I/O operations.

1.4.2 Multiprogramming Operating System

- **Definition:** Multiple programs reside in the main memory simultaneously, and the CPU switches between them so that the CPU is rarely idle.
- **Working Principle:** When one program waits for an I/O operation, the CPU picks another ready program from memory to execute.
- **Memory Structure:** [OS | Job 1 | Job 2 | Job 3]
- **CPU Flow:** Job 1 (I/O wait) -> Job 2 runs -> Job 1 resumes...
- **Advantage:** Increases overall CPU utilization.
- **Disadvantage:** CPU scheduling and memory management become complex.

1.4.3 Multitasking Operating System

- **Definition:** The ability of an OS to execute more than one task (process) seemingly simultaneously by rapidly switching the CPU between them (time-sharing at the user level).
- **Key Difference:** Multiprogramming focuses primarily on CPU utilization, whereas multitasking focuses on user interaction with several tasks at once (e.g., listening to music while typing a document).

1.4.4 Multiprocessing Operating System

- **Definition:** An OS that utilizes two or more CPUs (processors) within a single computer system to execute multiple processes simultaneously, achieving true parallelism.
- **Types:**
 - *Symmetric Multiprocessing (SMP):* All processors are treated equally and share the same main memory.
 - *Asymmetric Multiprocessing (AMP):* One master processor controls the system and schedules tasks, while the remaining processors follow its instructions.
- **Advantage:** Increased throughput and higher system reliability (if one CPU fails, the others continue working).
- **Disadvantage:** Highly complex OS design and increased hardware cost.

1.4.5 Time-Sharing Operating System

- **Definition:** A form of multiprogramming where CPU time is divided into small units called time slices or quanta. Each user or process gets a turn in rotation, giving the illusion that each user has a dedicated CPU.
- **Example:** UNIX.
- **Advantage:** Quick response time, making it ideal for interactive users.
- **Disadvantage:** Complex scheduling requirements; requires strict data communication and security boundaries among users.

1.4.6 Real-Time Operating System (RTOS)

- **Definition:** An OS that guarantees the processing of data and events within a strict, predefined time limit (deadline).
- **Types:**
 - *Hard Real-Time OS:* Missing a deadline causes critical system failure (e.g., missile control systems, pacemakers).

- *Soft Real-Time OS*: Missing a deadline is undesirable but not fatal to the system (e.g., video streaming, online booking platforms).
- **Examples**: Air traffic control systems, robotics, medical imaging infrastructure.

1.4.7 Distributed Operating System

- **Definition**: Manages a group of independent, networked computers and makes them appear to the users as a single, unified system (transparency). System resources are shared across multiple physical machines.
- **Advantages**: Seamless resource sharing, increased reliability, and faster computation via load distribution.
- **Disadvantages**: Complex architecture design; a network failure can impact the entire system; securing the distributed environment is more difficult.

1.4.8 Network Operating System (NOS)

- **Definition**: Runs on a dedicated server and provides the server with the capability to manage data, users, groups, security, applications, and other networking functions over a Local Area Network (LAN).
- **Key Difference**: Unlike a Distributed OS, each computer in a network retains its own local OS and autonomy; users are fully aware of the distinct, multiple machines.
- **Examples**: Novell NetWare, Windows Server, Linux-based NOS.

Comparison Table: Types of OS

Type	Key Idea	Example
Batch	Jobs grouped together, no direct user interaction	Payroll systems
Multiprogramming	Multiple jobs in memory, CPU is kept busy	Early UNIX
Multitasking	User runs multiple tasks concurrently	Windows
Multiprocessing	Multiple CPUs, true parallel execution	Modern servers
Time-Sharing	CPU divided into time slices among users	UNIX

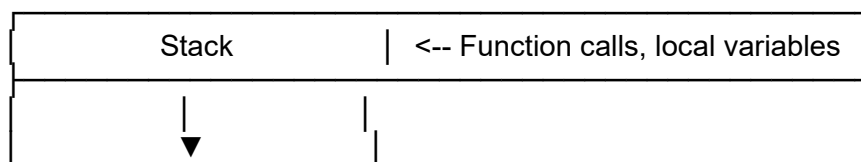
Type	Key Idea	Example
Real-Time (RTOS)	Processing bound by strict deadlines	VxWorks, pacemakers
Distributed	Multiple independent computers appear as one	Google's systems
Network OS	Manages resources over a LAN via a server	Windows Server

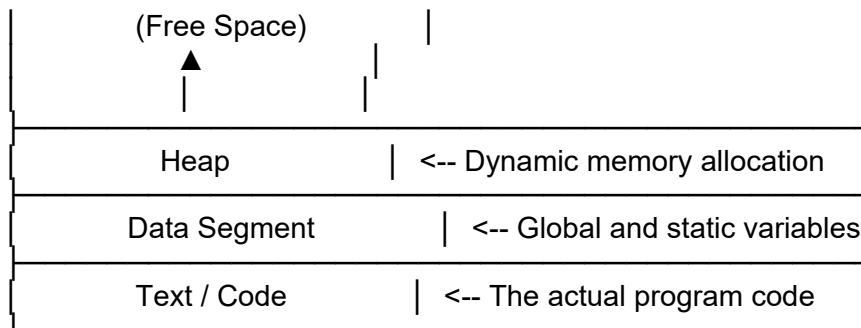
1.5 Program vs Process

Program	Process
A passive entity consisting of a set of instructions stored on a disk (e.g., an .exe file).	An active entity representing a program currently in execution.
Does not dynamically consume execution resources like CPU time.	Requires CPU time, memory space, and I/O access for execution.
Static in nature.	Dynamic in nature.
Exists permanently on the disk until deleted.	Exists for a limited time (from creation to termination).
Example: notepad.exe sitting on a hard drive.	Example: Notepad running on screen with an open file.

- **Definition of a Process:** A process is a program in execution, along with its current activity represented by the value of the Program Counter (PC), CPU registers, and variables.

A Process Layout in Memory



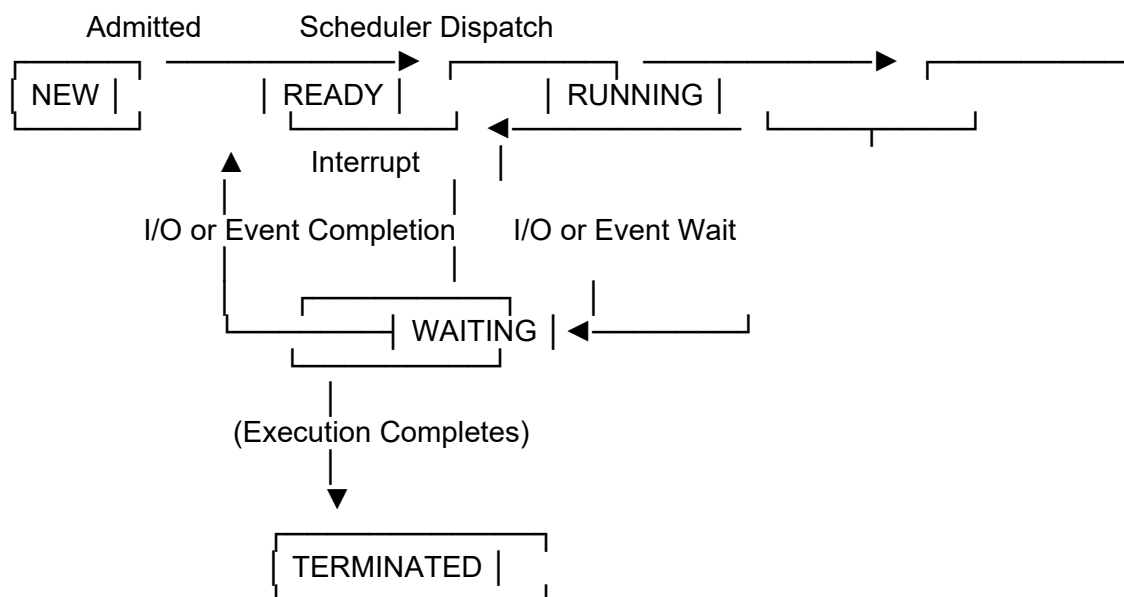


1.6 Process States

As a process executes, it moves through different states. The five common states are:

1. **New**: The process is currently being created.
2. **Ready**: The process is waiting to be assigned to a processor (stored in the ready queue).
3. **Running**: Instructions are actively being executed by the CPU.
4. **Waiting / Blocked**: The process is waiting for an external event to occur (such as I/O completion or receiving a signal).
5. **Terminated**: The process has completely finished execution.

Process State Diagram



💡 **Memory Trick:** N-R-R-W-T → "New Restaurant Runs Well Today" (New, Ready, Running, Waiting, Terminated).

1.7 Process Control Block (PCB)

- **Definition:** A PCB (also called a Task Control Block) is a data structure maintained by the OS for every individual process. It contains all the vital information needed to manage, track, and control the

process.

Contents of a PCB

Field	Description
Process ID (PID)	A unique identification number assigned to the process.
Process State	Current state of the process (New, Ready, Running, Waiting, Terminated).
Program Counter (PC)	Contains the address of the next instruction to be executed for this process.
CPU Registers	State of registers including accumulators, index registers, and general-purpose registers.
CPU Scheduling Info	Process priority value and pointers to scheduling queues.
Memory Management Info	Base and limit registers, page tables, or segment tables.
Accounting Info	Total CPU time used, time limits, and execution tracking numbers.
I/O Status Info	List of I/O devices allocated to the process and lists of open files.

Process ID (PID)	
Process State	
Program Counter	
CPU Registers	
Memory Limits	
I/O Status Info	

Accounting Info	
-----------------	--

 **Exam Tip:** PCB is frequently asked as a 2-mark or 5-mark question: "List and explain the contents of a PCB." Memorize the 8 fields shown above.

1.8 Process Life Cycle

The process life cycle refers to the complete journey of a process from creation to termination:

- **Creation:** The process is created by a system call like `fork()` (in UNIX systems) or by the OS at boot time. It is assigned a PID, and its PCB is initialized.
- **Scheduling:** The process moves into the Ready Queue, waiting for the OS scheduler to allocate CPU time.
- **Execution:** The CPU executes the process's instructions while it rests in the Running state.
- **Waiting:** If the process requests an I/O operation or resource that is not immediately available, it transitions into the Waiting state.
- **Termination:** Once the process finishes execution or is forced to stop, its allocated resources are released, and its PCB is deleted.
 - *Process Creation Details:* A parent process can spawn child processes. Resources may either be shared directly between the parent and child, or the child may receive its own distinct system allocations.
 - *Process Termination Reasons:* Normal completion, time limit exceeded, memory unavailable, errors (bounds violation, protection error), or parent termination causing cascading termination (child killed by the OS or parent via `abort()`).

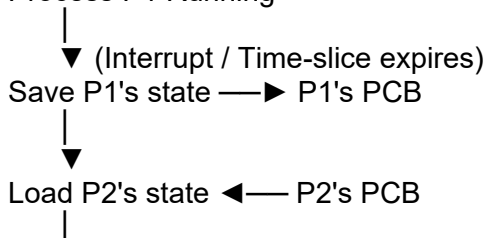
1.9 Context Switching

- **Definition:** Context Switching is the mechanism of saving the state (context) of a currently running process inside its PCB, and loading the saved state of another ready process so that its execution can be resumed from the exact point it was paused.
- **Why is it needed?** To allow multiple processes to share a single CPU efficiently, providing smooth multitasking capabilities.

Steps in a Context Switch

1. The OS saves the current state (PC, registers, progress) of the active process into its corresponding PCB.
2. The OS selects the next process from the Ready Queue using its scheduling algorithm.
3. The OS loads the saved state of the newly chosen process from its PCB back into the CPU registers.
4. Control is transferred to the new process, which resumes execution.

Process P1 Running





Process P2 Running

- **Important Performance Point:** Context switching is pure overhead. The system performs no useful application work while executing a switch. Excessive context switching degrades CPU efficiency, which is why context switch times must be kept to a minimum.
- **Causes of Context Switches:** Interrupts (I/O completion, timer ticks), multitasking time-slice expiry, priority-based preemption, or user/kernel mode switching.

CHAPTER-END QUESTIONS — UNIT I

2 Marks Questions

1. Define an Operating System.
2. What is multiprogramming?
3. Differentiate between a program and a process.
4. What is a PCB?
5. What is context switching?
6. Define a Real-Time Operating System.
7. What is a Time-Sharing OS?
8. Give two examples of a Network OS.
9. What is process termination?
10. Define multitasking.

5 Marks Questions

1. Explain the primary functions of an Operating System.
2. Explain the different process states using a well-labeled diagram.
3. What is a PCB? List and explain its contents in detail.
4. Differentiate between Multiprogramming, Multitasking, and Multiprocessing systems.
5. Explain a Distributed OS along with its advantages and disadvantages.
6. Explain the stages of the process life cycle.

10 Marks Questions

1. Explain in detail the different types of Operating Systems with relevant examples and diagrams.
2. Explain the concepts of a Process, Process States, and PCB in detail using structural diagrams.
3. What is context switching? Explain its absolute necessity and execution steps with a functional diagram.
4. Compare Batch OS, Time-Sharing OS, and Real-Time OS with real-world application examples.

Viva Questions (Unit I)

1. What is the kernel?
2. What is the structural difference between kernel mode and user mode?
3. Name any two real-time operating systems.
4. What system actions trigger a context switch?
5. What is the ready queue?
6. Is context switching considered useful work or overhead?
7. What is a zombie process?
8. What is an orphan process?

9. What is the core difference between SMP and AMP?
10. Give a real-life analogy for an Operating System.

Important PYQs (Unit I)

1. Explain the various types of Operating Systems. (10 marks — Repeated almost every year)
2. What is a PCB? Explain its structural components. (5 marks — Frequently asked)
3. Differentiate between a Process and a Program. (2/5 marks — Very common)
4. Explain Process states using a state transition diagram. (5/10 marks — Very common)
5. What is context switching? Why is it needed? (5 marks)

Unit I Summary

- An OS is the interface between a user and hardware; it manages the CPU, memory, files, devices, and security.
- Major OS types include Batch, Multiprogramming, Multitasking, Multiprocessing, Time-Sharing, RTOS, Distributed, and Network systems.
- A process is a program in active execution; it cycles through the New → Ready → Running → Waiting → Terminated states.
- A PCB stores vital execution tracking data for a process (PID, state, PC, registers, memory info, I/O info, accounting info).
- Context switching enables multitasking capabilities by saving and restoring process states; though essential, it represents pure computational overhead.

UNIT II — CPU SCHEDULING & DEADLOCK

PART A: CPU SCHEDULING

2.1 Introduction to CPU Scheduling

- **Definition:** CPU Scheduling is the process by which the OS decides which process in the Ready Queue is allocated CPU execution time next. Its primary goal is to maximize CPU utilization and optimize system throughput.
- **Why is it needed?** In multiprogramming systems, multiple processes reside in memory at once. Since there is generally a single CPU core, the OS must establish a systematic order in which processes run.
- **Types of Scheduling:**
 - *Preemptive Scheduling:* The CPU can be forcibly taken away from a running process before it finishes its current burst (e.g., SRTF, Round Robin, Preemptive Priority).
 - *Non-Preemptive Scheduling:* Once the CPU is allocated to a process, it cannot be preempted until it voluntarily relinquishes control by finishing execution or blocking for an I/O operation (e.g., FCFS, Non-Preemptive SJF).

2.2 Scheduling Criteria

Criterion	Meaning	Goal
CPU Utilization	Percentage of time the CPU is actively performing useful work	Maximize
Throughput	Number of individual processes completed per unit of time	Maximize
Turnaround Time (TAT)	Total elapsed time from process submission to its completion	Minimize
Waiting Time (WT)	Cumulative time a process spends waiting in the ready queue	Minimize
Response Time (RT)	Time from process submission to its very first CPU allotment	Minimize

Essential Numerical Formulas (MEMORIZE):

Completion Time (CT)=The exact timestamp at which a process finishes execution

Turnaround Time (TAT)=Completion Time (CT)–Arrival Time (AT)

Waiting Time (WT)=Turnaround Time (TAT)–Burst Time (BT)

Response Time (RT)=Timestamp of first CPU allotment–Arrival Time (AT)

💡 **Memory Trick:** "TAT walks after CT minus AT, WT walks after TAT minus BT"

2.3 FCFS (First Come First Serve)

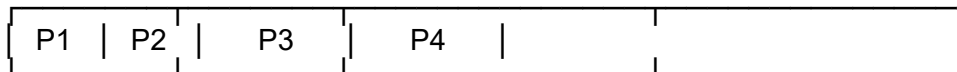
- **Theory:** Processes are executed strictly in the order they arrive in the ready queue. It is the simplest CPU scheduling algorithm available.
- **Working Principle:** Non-preemptive. Implemented using a First-In, First-Out (FIFO) queue structure.
- **Pseudocode:**
 1. Sort all incoming processes by their Arrival Time.
 2. Allocate the CPU to the process with the earliest Arrival Time.
 3. Once allocated, run the process to completion (no preemption allowed).
 4. Move sequentially to the next ready process in the queue.

Solved Example — FCFS

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Gantt Chart:

0 5 8 16 22



Calculation Table:

Process	AT	BT	CT	TAT (CT - AT)	WT (TAT - BT)
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

- Average Waiting Time = $\frac{0+4+6+13}{4} = 5.75$ ms
- Average Turnaround Time = $\frac{5+7+14+19}{4} = 11.25$ ms
- **Advantages:** Straightforward to understand, implement, and maintain; highly fair conceptually.
- **Disadvantages:** Suffers severely from the **Convoy Effect**, where short processes are forced to wait for extended periods behind a single long-running process, degrading average waiting times. Not optimal for interactive time-sharing systems.
- **Applications:** Basic batch compute environments with no distinct priority requirements.

2.4 SJF (Shortest Job First) — Non-Preemptive

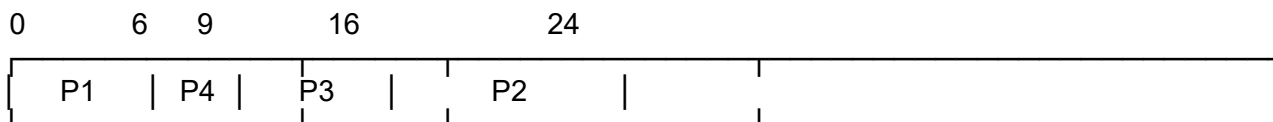
- **Theory:** The process containing the shortest CPU burst time is selected for execution first. In this non-preemptive variant, once a process takes the CPU, it runs until its burst is finished.
- **Pseudocode:**
 1. At any scheduling decision point, evaluate all processes that have arrived.
 2. Pick the process with the minimum burst time.
 3. Execute that process completely without interruption.
 4. Repeat the selection loop.

Solved Example — SJF (Non-Preemptive)

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	6
P2	1	8
P3	2	7
P4	3	3

- **Trace:**
 - At $t=0$, only P1 has arrived → run P1 from $t=0$ to $t=6$.
 - At $t=6$, available ready jobs are: P2 (BT=8), P3 (BT=7), and P4 (BT=3) → select P4 (smallest burst) → run P4 from $t=6$ to $t=9$.
 - At $t=9$, available ready jobs are: P2 (BT=8) and P3 (BT=7) → select P3 → run P3 from $t=9$ to $t=16$.
 - At $t=16$, only P2 is left → run P2 from $t=16$ to $t=24$.

Gantt Chart:



Calculation Table:

Process	AT	BT	CT	TAT	WT
P1	0	6	6	6	0

Process	AT	BT	CT	TAT	WT
P4	3	3	9	6	3
P3	2	7	16	14	7
P2	1	8	24	23	15

- Average Waiting Time= $\frac{40+3+7+15}{4}=13.75$ ms
- Average Turnaround Time= $\frac{46+6+14+23}{4}=22.25$ ms
- **Advantages:** Yields the minimum possible average waiting time among all non-preemptive algorithms (provably optimal for the non-preemptive category).
- **Disadvantages:** Requires advance knowledge of process burst times (which can typically only be estimated); long jobs can face **Starvation** if short jobs arrive continuously.
- **Applications:** Specialized batch systems where historical processing metrics make burst times predictable.

2.5 SRTF (Shortest Remaining Time First) — Preemptive SJF

- **Theory:** The preemptive variant of Shortest Job First. At any given moment, the process with the shortest remaining execution burst time is run. If a newly arrived process possesses a shorter remaining time than the currently executing process, the active process is preempted.
- **Provable Optimality:** SRTF yields the lowest possible average waiting time across all scheduling algorithms.

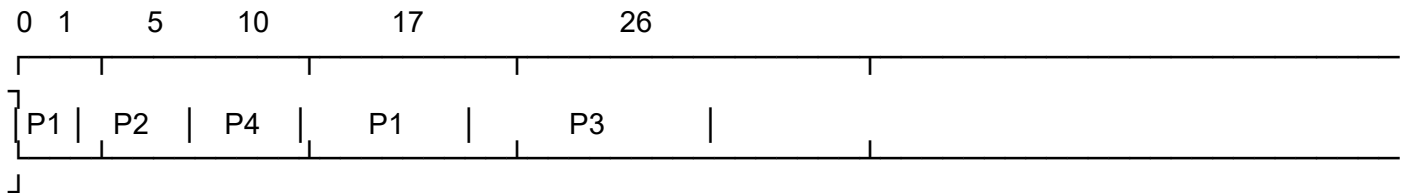
Solved Example — SRTF

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- **Step-by-Step Execution Trace:**
 - $t=0$: Only P1 is available → run P1. Remaining time for P1 = 8.
 - $t=1$: P2 arrives (BT=4). Remaining time for P1 is 7. Since P2's burst (4) < P1's remaining (7), **preempt P1** and run P2.

- $t=2$: P3 arrives (BT=9). P2's remaining time is 3. P2 remains the shortest → continue running P2.
- $t=3$: P4 arrives (BT=5). P2's remaining time is 2. P2 remains the shortest → continue running P2.
- $t=5$: P2 completely finishes execution (ran from $t=1$ to $t=5$). Now compare remaining times: P1 (7), P3 (9), P4 (5). P4 is the shortest → run P4.
- $t=10$: P4 completely finishes execution (ran from $t=5$ to $t=10$). Compare remaining times: P1 (7), P3 (9). P1 is the shortest → run P1.
- $t=17$: P1 completely finishes execution (ran from $t=10$ to $t=17$). Only P3 is left (remaining burst 9) → run P3.
- $t=26$: P3 finishes execution.

Gantt Chart:



Calculation Table:

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

- Average Waiting Time = $\frac{49+0+15+2}{4} = 6.5$ ms
- Average Turnaround Time = $\frac{417+4+24+7}{4} = 13$ ms
- **Advantages:** Achieves optimal average waiting times.
- **Disadvantages:** Incurs substantial management overhead due to frequent context switching; can cause continuous starvation for long-burst processes; hard to implement due to monitoring remaining burst timings.
- **Applications:** Time-critical systems where shorter computational jobs require immediate handling.

2.6 Priority Scheduling

- **Theory:** Each process is assigned an explicit priority value. The CPU is allocated to the ready process containing the highest priority. (Note: Typically, a lower numeric value signifies a higher priority status). It can be implemented as either preemptive or non-preemptive.

- **Core Practical Problem: Starvation** — Low-priority processes may wait indefinitely if higher priority tasks are constantly introduced.
- **Solution: Aging** — Gradually increment the priority value of processes that have spent an extended duration waiting in the ready queue.

Solved Example — Priority (Non-Preemptive)

(Note: Lower priority number = Higher priority status)

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	4	2
P2	1	3	1
P3	2	1	3
P4	3	5	4

- **Trace:**
 - $t=0$: Only P1 has arrived → run P1 from $t=0$ to $t=4$.
 - $t=4$: Available processes are: P2 (priority 1), P3 (priority 3), and P4 (priority 4). Select P2 due to highest priority status → run P2 from $t=4$ to $t=7$.
 - $t=7$: Available processes are: P3 (priority 3) and P4 (priority 4). Select P3 → run P3 from $t=7$ to $t=8$.
 - $t=8$: Only P4 is left → run P4 from $t=8$ to $t=13$.

Gantt Chart:



Calculation Table:

Process	AT	BT	CT	TAT	WT
P1	0	4	4	4	0

Process	AT	BT	CT	TAT	WT
P2	1	3	7	6	3
P3	2	1	8	6	5
P4	3	5	13	10	5

- Average Waiting Time= $\frac{40+3+5+5}{4}=13=3.25$ ms
- **Advantages:** High-priority infrastructure tasks can be given fast execution precedence; flexible operational design.
- **Disadvantages:** Prone to process starvation; requires the operational inclusion of aging techniques to manage low-priority workflows.
- **Applications:** Operating system internal functions where real-time system alerts take immediate precedence over user tasks.

2.7 Round Robin (RR) Scheduling

- **Theory:** Designed specifically for time-sharing systems. Each process is allocated a fixed time quantum (slice). The ready queue is treated as a circular list. The CPU cycles through the ready queue, executing each process for up to one time quantum. If a process does not finish within its allocated time slice, it is preempted and appended to the back of the ready queue.
- *This remains the most frequently encountered scheduling numerical problem in standard academic exams.*

Solved Example — Round Robin (Time Quantum = 4)

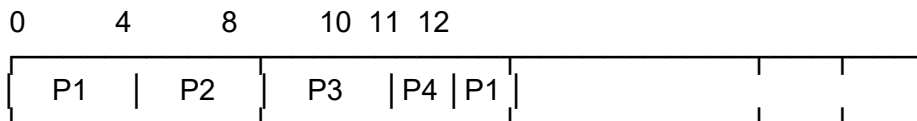
Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	4
P3	2	2
P4	3	1

- **Detailed Queue Execution Trace:**
 - $t=0$: Ready Queue = [P1]. Run P1 for 4 units ($t=0$ to $t=4$). Remaining burst for P1 = 1.
 - *During this execution window ($t=1, t=2, t=3$), processes P2, P3, and P4 arrive.* The queue updates sequentially to: [P2, P3, P4]. Then, the preempted P1 is appended to the back → Ready Queue

= [P2, P3, P4, P1].

- $t=4$: Fetch P2 from front. Run P2 for 4 units ($t=4$ to $t=8$). Remaining burst for P2 = 0 (finished). Ready Queue = [P3, P4, P1].
- $t=8$: Fetch P3 from front. Run P3 for its remaining 2 units ($t=8$ to $t=10$). Remaining burst for P3 = 0 (finished). Ready Queue = [P4, P1].
- $t=10$: Fetch P4 from front. Run P4 for its remaining 1 unit ($t=10$ to $t=11$). Remaining burst for P4 = 0 (finished). Ready Queue = [P1].
- $t=11$: Fetch P1 from front. Run P1 for its remaining 1 unit ($t=11$ to $t=12$). Remaining burst for P1 = 0 (finished).

Gantt Chart:



Calculation Table:

Process	AT	BT	CT	TAT	WT
P1	0	5	12	12	7
P2	1	4	8	7	3
P3	2	2	10	8	6
P4	3	1	11	8	7

- Average Waiting Time = $\frac{4+7+3+6+7}{5} = 5.75$ ms
- Average Turnaround Time = $\frac{4+12+7+8+8}{5} = 8.75$ ms
- **Advantages:** Highly equitable allocation of resource time; optimized interactive response time; perfect for multi-user interfaces.
- **Disadvantages:** System performance relies heavily on the selection of the time quantum. If the quantum is excessively large, RR degrades into FCFS scheduling. If it is too small, context switching overhead dominates the system.

2.8 Comparison Table: CPU Scheduling Algorithms

Algorithm	Preemptive?	Starvation Risk?	Best Suited For	Architecture Complexity
FCFS	No	No	Simple batch workflows	Very Simple
SJF (Non-Prem)	No	Yes (Long jobs)	Batch with estimated bursts	Medium
SRTF	Yes	Yes	Critical short-burst tasks	High
Priority	Either	Yes	High precedence tasks	Medium
Round Robin	Yes	No	Time-sharing / interactive systems	Medium

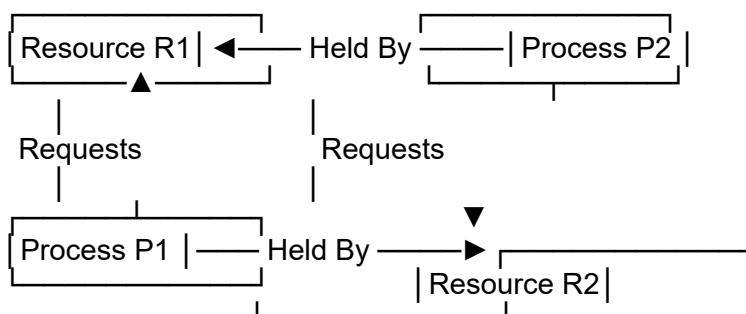
💡 **Memory Trick:** "Foolish Students Study Papers Regularly" → FCFS, SJF, SRTF, Priority, RR.

PART B: DEADLOCK

2.9 Deadlock — Definition

- **Definition:** A deadlock occurs when a set of processes are blocked permanently because each process holds a resource and waits for another resource held by another process in the same set. As a result, none of the processes can proceed.

Deadlock Dependency Diagram



- **Operational State:** P1 holds R2 and requests R1; P2 holds R1 and requests R2 → System is in a state of **Deadlock**.

2.10 Necessary Conditions for Deadlock (Coffman Conditions)

A deadlock can occur if and only if all four of the following conditions hold simultaneously:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one process can use the resource at a time).
2. **Hold and Wait:** A process must currently hold at least one resource and be waiting to acquire additional resources that are being held by other processes.
3. **No Preemption:** Resources cannot be forcibly taken from a process; they can only be released voluntarily by the process holding them after it has completed its task.
4. **Circular Wait:** A closed chain of processes exists such that each process holds one or more resources needed by the next process in the chain. For a set of processes $\{P_1, P_2, \dots, P_n\}$, P_1 waits for a resource held by P_2 , P_2 waits for P_3 , and P_n waits for P_1 .

💡 **Memory Trick: MHNC** → "My House Needs Cleaning" (Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait).

2.11 Resource Allocation Graph (RAG)

- **Definition:** A directed graph used to visually represent the current structural allocation state of system resources among distinct processes.
- **Graph Notation Values:**
 - **Process:** Represented as a Circle (P_1, P_2).
 - **Resource:** Represented as a Rectangle (R_1, R_2) with inner dots indicating the number of available instances.
 - **Request Edge:** A directed arrow pointing from Process to Resource ($P_i \rightarrow R_j$).
 - **Assignment Edge:** A directed arrow pointing from Resource instance to Process ($R_j \rightarrow P_i$).
- **Deadlock Graph Rules:**
 - If the RAG contains **no cycles**, the system is definitively free of deadlocks.
 - If the RAG contains a **cycle** and resources contain **only single instances**, a deadlock exists.
 - If multi-instance resources are present within a cycle, a deadlock is possible but not certain.

2.12 Deadlock Prevention

- **Strategy:** Prevent deadlocks by ensuring that at least one of the four necessary Coffman conditions cannot hold.

Target Condition	Prevention Methodology
Mutual Exclusion	Make resources shareable where possible (e.g., read-only files; though often impossible for single-user items like

Target Condition	Prevention Methodology
	printers).
Hold and Wait	Require processes to request all required resources at once, before execution begins. A process cannot hold any resources while requesting a new one.
No Preemption	If a process holding resources requests another resource that cannot be immediately allocated, all its current resources are preempted and released.
Circular Wait	Impose a strict global ordering on all resource types. Processes must request resources only in a strictly increasing order of enumeration.

- **System Trade-off:** Deadlock prevention results in low resource utilization and reduced system throughput due to its highly conservative nature.

2.13 Deadlock Avoidance

- **Strategy:** The OS dynamically evaluates every resource allocation request before granting it. The request is only approved if the resulting system state is guaranteed to be safe.
- **Safe State Concept:** A state is safe if the system can allocate resources to each process (up to its maximum claim) in some order and still avoid deadlock. This ordering is known as a **Safe Sequence**.

The Banker's Algorithm

Developed by Edsger Dijkstra, this algorithm is called the Banker's Algorithm because it is modeled on a banking system that ensures cash is never allocated in a way that fails to satisfy the needs of all its customers.

Data Structures Utilized:

- $Available[m]$: A vector of length m indicating the number of available instances of each resource type.
- $Max[n][m]$: A matrix defining the maximum resource demand of each process.
- $Allocation[n][m]$: A matrix defining the resource instances currently allocated to each process.
- $Need[n][m]$: A matrix defining the remaining resources required by each process:
 $Need[i][j] = Max[i][j] - Allocation[i][j]$

Safety Algorithm Steps:

1. Let $Work = Available$ and $Finish[i] = false$ for all processes $i = 0, 1, \dots, n-1$.
2. Search for a process index i that satisfies both:
 $Finish[i] == false$ AND $Need[i] \leq Work$
3. If such a process i is found, update the vectors as follows:
 $Work = Work + Allocation[i]$
 $Finish[i] = true$
Return to Step 2.
4. If no such process exists:
 - If $Finish[i] == true$ for all i , the system is in a **Safe State**.
 - Otherwise, the system is in an **Unsafe State** (potential deadlock).

Solved Example — Banker's Algorithm

Assume 5 processes ($P_0 - P_4$) and 3 resource types (A, B, C) with total system instances: $A=10, B=5, C=7$.

Allocation Matrix:

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max Demand Matrix:

Process	A	B	C
P0	7	5	3
P1	3	2	2

Process	A	B	C
P2	9	0	2
P3	2	2	2
P4	4	3	3

Step 1: Calculate the Need Matrix (Max-Allocation)

- Need for P0 = $(7-0, 5-1, 3-0) = (7, 4, 3)$
- Need for P1 = $(3-2, 2-0, 2-0) = (1, 2, 2)$
- Need for P2 = $(9-3, 0-0, 2-2) = (6, 0, 0)$
- Need for P3 = $(2-2, 2-1, 2-1) = (0, 1, 1)$
- Need for P4 = $(4-0, 3-0, 3-2) = (4, 3, 1)$

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Step 2: Calculate the Initial Available Vector

Sum of Allocations = $(0+2+3+2+0, 1+0+0+1+0, 0+0+2+1+2) = (7, 2, 5)$

Available = Total Instances - Sum of Allocations = $(10-7, 5-2, 7-5) = (3, 3, 2)$

Step 3: Apply the Safety Testing Algorithm

- Initialize: Work = $(3, 3, 2)$
- Test P₀: Is Need $(7, 4, 3) \leq$ Work $(3, 3, 2)$? No. (P₀ must wait).
- Test P₁: Is Need $(1, 2, 2) \leq$ Work $(3, 3, 2)$? Yes.
 - Work = $(3, 3, 2) +$ Allocation P₁ $(2, 0, 0) = (5, 3, 2)$

- $Finish[P_1]=true$
- Test P_2 : Is $Need(6,0,0) \leq Work(5,3,2)$? No.
- Test P_3 : Is $Need(0,1,1) \leq Work(5,3,2)$? Yes.
 - $Work=(5,3,2)+Allocation\ P_3(2,1,1)=(7,4,3)$
 - $Finish[P_3]=true$
- Test P_4 : Is $Need(4,3,1) \leq Work(7,4,3)$? Yes.
 - $Work=(7,4,3)+Allocation\ P_4(0,0,2)=(7,4,5)$
 - $Finish[P_4]=true$
- Retest P_0 : Is $Need(7,4,3) \leq Work(7,4,5)$? Yes.
 - $Work=(7,4,5)+Allocation\ P_0(0,1,0)=(7,5,5)$
 - $Finish[P_0]=true$
- Retest P_2 : Is $Need(6,0,0) \leq Work(7,5,5)$? Yes.
 - $Work=(7,5,5)+Allocation\ P_2(3,0,2)=(10,5,7)$
 - $Finish[P_2]=true$

All processes successfully completed. The system is in a **Safe State**.

Safe Sequence: $\langle P_1, P_3, P_4, P_0, P_2 \rangle$

2.14 Deadlock Detection

- **Strategy:** Allow deadlocks to occur, periodically invoke a detection algorithm to identify them, and then initiate a system recovery process.
- **Detection Algorithm Steps:** Uses a framework similar to the Banker's safety test. It operates directly on the *Allocation* matrix, an explicit *Request* matrix, and the *Available* vector.
 1. Initialize $Work=Available$. For all processes, if $Allocation[i] \neq 0$, set $Finish[i]=false$; else, set $Finish[i]=true$.
 2. Find a process index i such that $Finish[i]==false$ and $Request[i] \leq Work$.
 3. If found, update the vectors: $Work=Work+Allocation[i]$, set $Finish[i]=true$, and repeat Step 2.
 4. If there is any process i for which $Finish[i]==false$, then that process i is **deadlocked**.

2.15 Deadlock Recovery

When a deadlock is detected, the system recovers using one of two primary methods:

1. **Process Termination:**
 - *Abort all deadlocked processes:* Simple but costly, as all partial computation work within those processes is lost.
 - *Abort processes one at a time:* The detection algorithm is rerun after each termination until the deadlock cycle is broken.
2. **Resource Preemption:**
 - Forcibly reclaim resources from one or more processes and allocate them to other deadlocked processes until the cycle is broken.
 - *Key considerations:* Choosing a victim process (minimizing cost), rolling back the process to a previous safe checkpoint, and preventing starvation (ensuring the same process is not repeatedly targeted).

Comparison Table: Deadlock Handling Approaches

Approach	Assessment Time	Resource Utilization	Computational Overhead
Prevention	Static (design time)	Low	Low
Avoidance	Dynamic (at each request)	Medium	Medium-High
Detection & Recovery	Periodic (after execution)	High	High (during recovery)

CHAPTER-END QUESTIONS — UNIT II

2 Marks Questions

1. Define CPU Scheduling.
2. What is Turnaround Time?
3. Define Deadlock.
4. What are the four necessary Coffman conditions for a deadlock?
5. What is the Convoy Effect?
6. What is a safe state?
7. Define Starvation and Aging.
8. What is a Resource Allocation Graph?
9. What is the function of the time quantum in Round Robin scheduling?
10. What is the fundamental difference between deadlock prevention and deadlock avoidance?

5 Marks Questions

1. Explain FCFS scheduling using a custom example.
2. Explain Round Robin scheduling using a solved numerical problem.
3. Explain the four necessary conditions for a deadlock to occur.
4. Explain the design structures of a Resource Allocation Graph.
5. Explain standard methodologies for deadlock prevention.
6. Explain the step-by-step execution of the deadlock detection algorithm.

10 Marks Questions

1. Explain and solve numerical problems comparing FCFS, SJF, and Round Robin scheduling algorithms.
2. Explain the Banker's Algorithm in detail using a solved numerical matrix example.
3. Provide a comprehensive analysis of Deadlocks: definition, necessary conditions, prevention, avoidance, detection, and recovery mechanisms.

Viva Questions (Unit II)

1. Which scheduling algorithm gives the minimum average waiting time?
2. What is the operational effect of an excessively small time quantum in Round Robin?
3. Why does Shortest Job First scheduling cause process starvation?

4. What is the exact operational difference between SJF and SRTF?
5. Who proposed the Banker's Algorithm?
6. What is the structural difference between the Need and Max matrices?
7. Can a cycle exist in a Resource Allocation Graph without indicating a deadlock?
8. Name the four Coffman conditions.
9. What is the difference between preemptive and non-preemptive scheduling?
10. What is a safe sequence?

Important PYQs (Unit II)

1. Solve scheduling numericals to calculate average waiting time and turnaround time. (10 marks — *Appears almost every year*)
2. Explain the Banker's Algorithm with a practical matrix example. (10 marks — *Very frequently asked*)
3. What are the necessary conditions for a deadlock? Explain each. (5 marks)
4. Differentiate between deadlock prevention, avoidance, and detection. (5 marks)

Unit II Summary

- CPU Scheduling determines how ready processes are allocated execution time. Key metrics include CPU utilization, throughput, turnaround time, waiting time, and response time.
- Algorithms include FCFS (prone to the convoy effect), SJF/SRTF (optimal for minimizing waiting time but prone to starvation), Priority (requires aging), and Round Robin (allocates time equitably using time slices).
- A deadlock requires four conditions to occur simultaneously: Mutual Exclusion, Hold & Wait, No Preemption, and Circular Wait.
- The Banker's Algorithm avoids deadlocks by dynamically evaluating requests and only granting allocations that keep the system in a safe state.

UNIT III — MEMORY MANAGEMENT

3.1 Introduction

- **Definition:** Memory Management is the core OS function responsible for allocating and deallocating primary memory (RAM) space to processes, tracking which parts of memory are currently in use, and ensuring processes remain isolated to prevent mutual interference.

3.2 Contiguous Memory Allocation

- **Definition:** Each process is allocated a single, unbroken block of contiguous memory locations.
- **Techniques:**
 - *Fixed (Static) Partitioning:* Memory is divided into fixed-size regions at boot time. Each partition can hold exactly one process, regardless of its size.
 - *Dynamic Partitioning:* Partitions are created dynamically in memory to match the exact size requirements of incoming processes.

Operating System	
Process P1	
Process P2	

(Free Space)	
Process P3	

Partition Allocation Strategies:

- **First Fit:** Allocates the very first free block encountered from the beginning of memory that is large enough to satisfy the request.
- **Best Fit:** Searches the entire memory space to allocate the smallest available free block that is large enough, minimizing leftover space.
- **Worst Fit:** Allocates the largest available free block in memory, leaving behind a large leftover hole.

💡 **Memory Trick:** *First Fit = Fastest to find; Best Fit = Minimizes immediate wastage but is slow; Worst Fit = Leaves the largest leftover hole.*

3.3 Fragmentation

- **Definition:** Fragmentation is the condition where available memory space is wasted because it is broken into small, non-contiguous pieces.
- **Types:**
 - **External Fragmentation:** Total free memory space is sufficient to satisfy a process request, but the space is split into small holes scattered across memory. As a result, a process cannot be allocated because a single, contiguous block is unavailable.
 - **Solution: Compaction** (shuffling the contents of memory to consolidate all free space into one large block).
 - **Internal Fragmentation:** Memory is allocated in fixed-size blocks. When a process receives a block that is slightly larger than its requested size, the unused space inside that allocated block is wasted.

External Fragmentation Visual:

[Process P1] [Free 4KB] [Process P2] [Free 8KB] <-- Total free space = 12KB, but scattered.

Internal Fragmentation Visual:

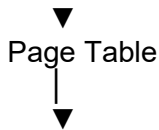
Process Memory 18KB	Wasted 2KB Space	<-- Inside a fixed 20KB memory partition block.
---------------------	------------------	---

3.4 Paging

- **Definition:** Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory.
- **Working Principle:**
 - Physical memory is broken down into fixed-size blocks called **Frames**.
 - Logical (process) memory is broken down into blocks of the exact same size called **Pages**.
 - Logical addresses generated by the CPU are split into a **Page Number (p)** and a **Page Offset (d)**.
 - A **Page Table** maps the logical page numbers to their corresponding physical frame numbers.

Logical Address: [Page Number (p) | Offset (d)]

|



Physical Address: [Frame Number (f) | Offset (d)]

Address Translation Formula:

Physical Address=(Frame Number×Page Size)+Offset

Solved Example — Paging

Assume Page Size=1 KB (1024 bytes). A process generates a Logical Address=3200.

- Page Number= $\frac{3200}{1024}=3$ (Integer quotient)
- Offset= $3200 \pmod{1024}=128$ (Remainder)
- If Page 3 maps directly to Frame 6 in the page table:
Physical Address= $(6 \times 1024) + 128 = 6144 + 128 = 6272$
- **Advantages:** Completely eliminates external fragmentation; simplifies memory allocation since any free frame can be assigned.
- **Disadvantages:** Introduces internal fragmentation (typically on the final page of a process); requires an extra memory access loop to query the page table. This delay is minimized using a **Translation Lookaside Buffer (TLB)**, which acts as a fast hardware cache for page table entries.

3.5 Segmentation

- **Definition:** Segmentation is a memory management scheme that aligns with the programmer's view of memory. Instead of a linear sequence of bytes, a program is divided into variable-sized logical blocks called **Segments** (e.g., Main Program, Functions, Objects, Stack, Global variables).
- **Working Principle:**
 - Logical addresses consist of a **Segment Number (s)** and an **Offset (d)**.
 - A **Segment Table** tracks the allocation using two metrics: **Base** (the starting physical address) and **Limit** (the exact physical length of the segment).

Segment Address Translation:

If Offset (d) < Limit, then:

Physical Address = Base + Offset

Else:

Generate a Segmentation Trap (Address Error)

Paging vs Segmentation

Basis	Paging	Segmentation
Block Allocation Size	Fixed-size pages / frames	Variable-size segments

Basis	Paging	Segmentation
System Perspective	Physical breakdown of memory	Logical / Programmer's view
Fragmentation Type	Internal fragmentation	External fragmentation
Address Components	Page Number + Offset	Segment Number + Offset
User Visibility	Completely invisible to the user	Visible and managed via code structures

3.6 Paging with Segmentation (Segmented Paging)

- **Definition:** A hybrid scheme where segments are divided into pages. This combines the structural benefits of segmentation with the fragmentation advantages of paging.
- **Address Format Structure:** (Segment Number, Page Number, Offset)
Logical Address → Segment Table → Target Segment Page Table → Physical Frame Number

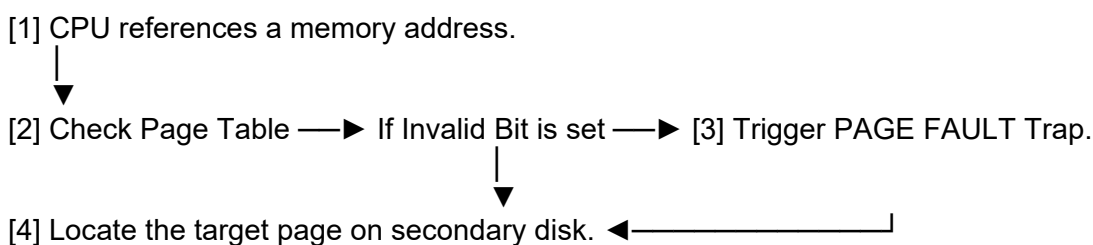
3.7 Virtual Memory

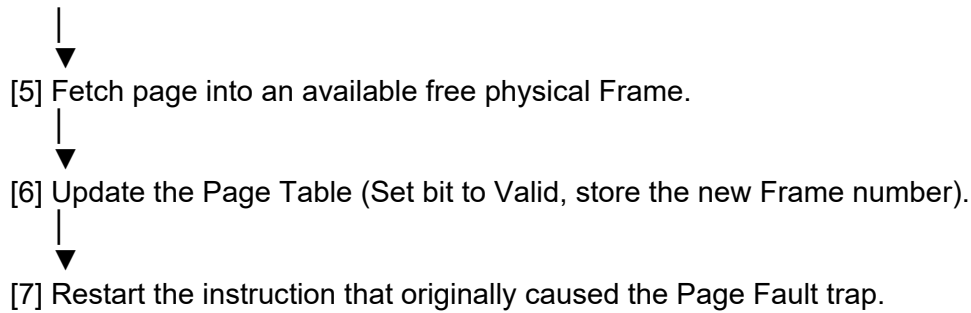
- **Definition:** Virtual Memory is a technique that allows a process to execute even if it is not entirely present in physical memory. It provides the illusion of a very large primary memory by using secondary storage space as an extension of RAM.
- **Why is it needed?** It allows programs larger than physical RAM to execute, increases the degree of multiprogramming by fitting more processes into memory, and speeds up process initialization since files do not need to be loaded fully before execution starts.

3.8 Demand Paging

- **Definition:** An implementation of virtual memory where pages are loaded into primary memory only when they are referenced (demanded) during execution, rather than being loaded all at once at startup.
- **Working Principle:** Each page table entry includes a **Valid / Invalid Bit**. If a process attempts to access a page marked as invalid, a **Page Fault** occurs, triggering the OS to retrieve the page from disk.

Page Fault Handling Steps:





3.9 Page Replacement Algorithms

When a page fault occurs and there are no free frames available in memory, the OS must select an existing page to remove (replace) to make room for the incoming page.

3.9.1 FIFO (First In First Out)

- **Theory:** Replaces the page that has been in memory the longest, regardless of how frequently or recently it has been accessed.
- **Belady's Anomaly:** In FIFO scheduling, increasing the number of physical frames available can sometimes result in an *increased* number of page faults. This counter-intuitive behavior is unique to FIFO and does not occur in stack-based algorithms like LRU.

Solved Example — FIFO

Reference String: 1, 3, 0, 3, 5, 6, 3 allocated across 3 frames.

Reference	Frame 1	Frame 2	Frame 3	Result
1	1	-	-	Page Fault
3	1	3	-	Page Fault
0	1	3	0	Page Fault
3	1	3	0	Page Hit
5	5	3	0	Page Fault (1 replaced; oldest)
6	5	6	0	Page Fault (3

Reference	Frame 1	Frame 2	Frame 3	Result
				<i>replaced; oldest)</i>
3	5	6	3	Page Fault (<i>0 replaced; oldest)</i>

- **Total Page Faults: 6 | Total Hits: 1**

3.9.2 LRU (Least Recently Used)

- **Theory:** Replaces the page that has not been accessed for the longest duration, using past reference history to predict future access patterns.

Solved Example — LRU

Reference String: 1, 3, 0, 3, 5, 6, 3 allocated across 3 frames.

Reference	Frames State (MRU → LRU)	Result
1	[1]	Page Fault
3	[3, 1]	Page Fault
0	[0, 3, 1]	Page Fault
3	[3, 0, 1] (<i>3 accessed, moved to front</i>)	Page Hit
5	[5, 3, 0] (<i>1 replaced; least recently used</i>)	Page Fault
6	[6, 5, 3] (<i>0 replaced; least recently used</i>)	Page Fault
3	[3, 6, 5] (<i>3 accessed, moved to front</i>)	Page Hit

- **Total Page Faults: 5 | Total Hits: 2**

3.9.3 Optimal Page Replacement (OPT / MIN)

- **Theory:** Replaces the page that will not be used for the longest duration in the future. This algorithm provides the lowest possible page fault rate for a fixed number of frames. However, because it requires advance knowledge of the future reference string, it cannot be implemented in real-world systems and is used primarily as a theoretical benchmark.

Solved Example — Optimal

Reference String: 1, 3, 0, 3, 5, 6, 3 allocated across 3 frames.

Reference	Frames Array State	Result	Replacement Reason
1	[1]	Page Fault	
3	[1, 3]	Page Fault	
0	[1, 3, 0]	Page Fault	
3	[1, 3, 0]	Page Hit	
5	[5, 3, 0]	Page Fault	Page 1 is replaced because it is never referenced again.
6	[6, 3, 0]	Page Fault	Page 5 is replaced because it is not needed in the remaining window.
3	[6, 3, 0]	Page Hit	

- **Total Page Faults: 5** (*The minimum achievable rate*)

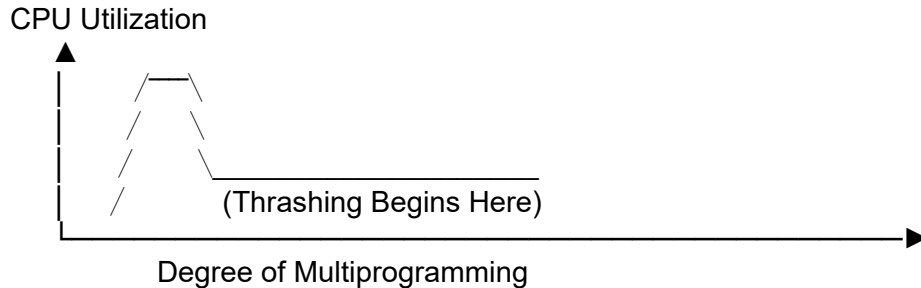
💡 **Memory Trick:** "FIFO forgets fairness, LRU looks back, Optimal only in dreams."

3.10 Thrashing

- **Definition:** Thrashing occurs when a system spends more time swapping pages in and out of

secondary storage than executing actual instructions. This leads to a severe drop in CPU utilization.

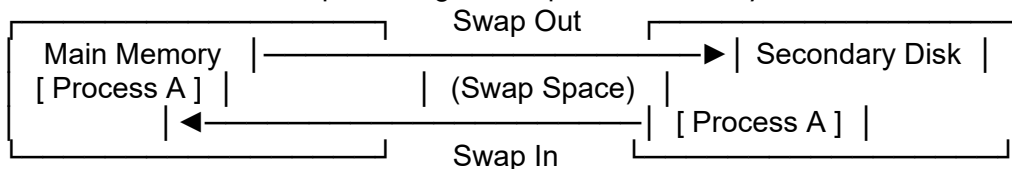
- **Cause:** This happens when the degree of multiprogramming is too high and the total number of allocated frames is less than the minimum required frames (the Working Set) for the active processes. As a result, processes continuously fault and stall each other.



- **Solutions:**
 - Implement the **Working Set Model** to ensure each process receives enough frames to match its localized allocation needs.
 - Monitor the **Page Fault Frequency (PFF)**. If the fault rate rises too high, allocate more frames to that process; if it drops too low, reclaim unused frames.
 - Reduce the degree of multiprogramming by suspending or terminating some active processes.

3.11 Swapping

- **Definition:** Swapping is a technique where an entire process is temporarily moved out of main memory to a backing store (swap space on disk) and later brought back into memory to resume execution. This helps manage and optimize memory utilization across active processes.



CHAPTER-END QUESTIONS — UNIT III

2 Marks Questions

1. Define Fragmentation.
2. Differentiate between internal and external fragmentation.
3. What is Paging?
4. What is a Page Table?
5. Define Virtual Memory.
6. What is Demand Paging?
7. Define Thrashing.
8. What is Swapping?
9. What is Belady's Anomaly?
10. What is a TLB?

5 Marks Questions

1. Explain Paging in detail using a clear operational diagram.
2. Explain Segmentation and how address resolution is achieved.

3. Differentiate between the functional characteristics of Paging and Segmentation.
4. Explain the FIFO page replacement algorithm using a custom trace example.
5. Explain the LRU page replacement algorithm using a structural reference table.
6. Explain the core causes of system Thrashing and outline its diagnostic solutions.

10 Marks Questions

1. Provide a comprehensive analysis of Paging, Segmentation, and their hybrid implementation (Segmented Paging).
2. Explain the architecture of Demand Paging and detail every stage of the Page Fault handling process.
3. Solve page replacement numericals comparing FIFO, LRU, and Optimal algorithms using the reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. Assume 3 frames.

Viva Questions (Unit III)

1. What architectural conditions cause internal fragmentation?
2. What is memory compaction?
3. What does the working set model track?
4. Which page replacement algorithm yields the absolute minimum page faults?
5. Does the LRU algorithm suffer from Belady's Anomaly?
6. What is a page fault?
7. What is the operational difference between paging and swapping?
8. Why is a TLB integrated into modern MMU hardware?
9. What does the valid/invalid bit indicate in a page table?
10. What is a memory frame?

Important PYQs (Unit III)

1. Explain Paging and Segmentation architectures with detailed memory flow diagrams. (10 marks — Very common)
2. Solve page replacement algorithm numericals comparing FIFO and LRU. (10 marks — Appears every year)
3. What is Thrashing? How can an operating system prevent it? (5 marks)

Unit III Summary

- Memory Management coordinates the allocation of physical memory. Contiguous allocation schemes partition memory using strategies like First Fit, Best Fit, or Worst Fit.
- Fragmentation can be internal (wasted space within a fixed allocation block) or external (scattered, non-contiguous blocks of free memory).
- Paging divides memory into fixed-size pages and frames to eliminate external fragmentation. Segmentation organizes memory into variable-sized logical units that reflect the programmer's view.
- Virtual memory uses demand paging to execute processes that are not fully loaded into RAM. Pages are brought into memory from disk only when a page fault occurs.
- Page replacement algorithms select which page to evict when memory is full. Options include FIFO (simple but subject to Belady's anomaly) and LRU (a practical algorithm based on recent history). Thrashing occurs when excessive page faults cause the system to spend more time paging than executing instructions.

UNIT IV — FILE SYSTEM

4.1 File Concept

- **Definition:** A file is a named collection of related information recorded on secondary storage. It represents the smallest logical unit of storage managed by the Operating System.
- **Examples:** Text documents (.txt), word files (.docx), executable application files (.exe), media items (.mp3, .jpg).

4.2 File Attributes

Every file is associated with metadata managed directly by the file system:

- **Name:** The symbolic file name stored in a human-readable format.
- **Identifier:** A unique numeric tag that identifies the file within the file system.
- **Type:** Specifies the file format for systems that support multiple file types (e.g., binary, executable, text).
- **Location:** A pointer to the device and the specific physical blocks where the file is stored.
- **Size:** The current size of the file, measured in bytes, kilobytes, or megabytes.
- **Protection:** Access-control metadata that defines who can read, write, or execute the file.
- **Time, Date, User ID:** Timestamps tracking creation, modification, and last access, along with the owner's identity.

💡 **Memory Trick:** NITLS-PT → Name, Identifier, Type, Location, Size, Protection, Time.

4.3 File Operations

The OS provides low-level system calls to perform basic operations on files:

- **Create:** Allocates necessary disk space and creates a corresponding entry in the directory structure.
- **Write:** Appends or overwrites data at the current write-pointer location within the file.
- **Read:** Retrieves data from the current read-pointer location within the file.
- **Reposition (Seek):** Moves the internal file pointer to a specific offset or location within the file.
- **Delete:** Removes the file's entry from the directory index and marks its storage blocks as free.
- **Truncate:** Clears the contents of the file while preserving its attributes and directory entry (resetting the file length to 0).
- **Open:** Copies the file's attributes and block locations from disk into main memory for fast access during operations.
- **Close:** Removes the file entry from the in-memory open-file table and flushes any pending writes to disk.

4.4 Directory Structure

- **Definition:** A directory is a structured container that maps human-readable file names to their corresponding system locations and metadata.

Types of Directory Structures:

- 1. Single-Level Directory:**
 - All files are stored in a single, flat directory.
 - *Limitation:* Prone to naming conflicts, as every file in the system must have a unique name, even if created by different users.
- 2. Two-Level Directory:**
 - The system creates a Master File Directory (MFD) containing separate User File Directories (UFD) for each user account.
 - *Limitation:* Resolves naming conflicts across different user accounts, but users cannot group or sub-categorize files within their own directory.
- 3. Tree-Structured (Hierarchical) Directory:**
 - Directories can contain both files and subdirectories, forming a tree structure. This is the model used by modern operating systems like Windows and Linux.
 - *Advantage:* Allows users to organize files logically into nested folders and subfolders.
- 4. Acyclic-Graph Directory:**
 - An extension of the tree structure that allows directories to share subdirectories and files via links (e.g., shortcuts or aliases). No cycles are permitted in the graph.
- 5. General Graph Directory:**
 - Allows cycles within the directory structure (e.g., a directory pointing back to an ancestor directory).
 - *Limitation:* Requires garbage collection to reclaim space and cycle-detection algorithms during directory traversals to prevent infinite loops.

Comparison Table: Directory Structures

Structure	File Sharing Capabilities	Naming Conflicts	Architecture Complexity
Single-Level	None	High	Extremely Simple
Two-Level	None across accounts	Isolated by user account	Simple
Tree-Structured	Limited	None	Moderate
Acyclic Graph	Highly flexible sharing	None	Complex
General Graph	Highly flexible sharing	None	High (Requires cycle handling)

4.5 File Allocation Methods

File allocation methods define how the operating system maps logical file blocks onto physical disk blocks.

4.5.1 Contiguous Allocation

- **Theory:** Each file is allocated a continuous, sequential block of sectors on the disk. For example, a file starting at block 2 with a length of 4 occupies blocks 2, 3, 4, and 5.
- **Advantages:** Excellent performance for both sequential and direct (random) access since all data blocks are physically adjacent. The directory entry only needs to store the starting block address and the total length.
- **Disadvantages:** Subject to severe external fragmentation. It can also be difficult to grow a file over time if there is no contiguous free space available immediately after its last block.

4.5.2 Linked Allocation

- **Theory:** Each file is stored as a linked list of disk blocks scattered across the storage medium. Every block contains a data payload along with a pointer to the next block in the sequence.
- **Advantages:** Eliminates external fragmentation entirely. Files can grow dynamically as long as there is at least one free block available on the disk.
- **Disadvantages:** Does not support efficient random or direct access; retrieving a specific block requires traversing the list sequentially from the start block. Pointers also consume a small amount of space within each block, and a single corrupted pointer can make the remainder of the file inaccessible.
 - *Variation — FAT (File Allocation Table):* This variant isolates block pointers into a centralized table stored at the beginning of the volume. This approach dramatically improves random access speeds by allowing the OS to traverse pointers in memory rather than reading individual data blocks from disk.

4.5.3 Indexed Allocation

- **Theory:** All block pointers for a given file are collected into a single location known as the **Index Block**. The index block contains an ordered array of pointers to the file's individual data blocks.

Index Block for File A:

Pointer to Data Block 5	
Pointer to Data Block 12	
Pointer to Data Block 93	
Pointer to Data Block 14	

- **Advantages:** Supports fast direct and random access while eliminating external fragmentation. Files can grow dynamically without requiring contiguous space.
- **Disadvantages:** Introduces storage overhead for the index block itself, which can be inefficient for small files. For very large files, a single index block may run out of pointer slots. This limitation is managed using mechanisms like linked index blocks, multi-level indexing, or combined indexing schemes (such as UNIX inodes).

Comparison Table: File Allocation Methods

Method	Access Support	Fragmentation Type	Growth Dynamics
Contiguous	Fast Direct + Sequential	External Fragmentation	Difficult
Linked	Sequential Access Only	None	Easy
Indexed	Fast Direct + Sequential	None (Index block overhead)	Easy

4.6 File Access Methods

- **Sequential Access:** Information in the file is processed in order, one record after another. This is the simplest access method and is used for tasks like reading a text file line by line.
- **Direct (Random) Access:** The file is treated as a numbered sequence of blocks or records. Applications can read or write blocks in any order by jumping directly to a specific target block. This method is crucial for database systems.
- **Indexed Access:** Implements an index on top of a direct-access foundation. The system searches the index to find a pointer to the target record, allowing rapid retrieval of specific data fields.

4.7 File Protection

- **Definition:** File protection mechanisms restrict access to files by ensuring that only authorized users can perform specific operations (such as read, write, or execute).

The Access Control List (ACL) Model (UNIX-style):

Permissions are defined for three distinct user classes:

1. **Owner:** The user who originally created the file.
2. **Group:** A collection of users who share common access requirements for the file.
3. **Others (World):** All other users on the system.

Each class is assigned three standard permission bits: **Read (r)**, **Write (w)**, and **Execute (x)**.

Example UNIX Permission Representation: `rwxr-xr--`

`rwX    r-X    r--`

Owner Group Others

- **Interpretation:** The Owner has full read, write, and execute permissions. Members of the Group can read and execute the file. Others have read-only access.
- **Alternative Protection Methods:** Explicit password protection on individual files, Access Control Lists (detailed matrices listing permissions for specific users), or cryptographically encrypting files to

prevent unauthorized access even if the physical storage media is compromised.

CHAPTER-END QUESTIONS — UNIT IV

2 Marks Questions

1. Define a File.
2. List any four common file attributes.
3. What is a directory?
4. Define Contiguous File Allocation.
5. What is the function of a File Allocation Table (FAT)?
6. Differentiate between sequential access and direct access methods.
7. What is an index block?
8. What is an Access Control List (ACL)?

5 Marks Questions

1. Explain the fundamental file operations managed by an operating system.
2. Explain the structural differences between Single-Level, Two-Level, and Tree-Structured directory models.
3. Explain Linked Allocation along with its primary advantages and disadvantages.
4. Explain the Indexed Allocation scheme using a structural diagram.
5. Explain standard file protection mechanisms.

10 Marks Questions

1. Provide a comprehensive analysis of directory structures, detailing their advantages, disadvantages, and structural layouts.
2. Explain Contiguous, Linked, and Indexed file allocation methods. Detail their functional characteristics, storage efficiency, and impact on system access speeds.

Viva Questions (Unit IV)

1. What is the structural difference between a file and a directory?
2. Which allocation method is used in UNIX inode architectures?
3. What causes external fragmentation in contiguous file allocation?
4. What is a symbolic link?
5. What does execute (x) permission mean when applied to a directory in UNIX?
6. Why does the linked allocation method fail to support efficient direct access?

Important PYQs (Unit IV)

1. Explain file allocation methods with detailed diagrams. *(10 marks — Very common)*
2. Explain hierarchical and graph-based directory structures. *(5/10 marks — Frequently asked)*
3. What is file protection? Explain access control classes. *(5 marks)*

Unit IV Summary

- A file is the fundamental logical unit of storage managed by the OS. It includes attributes such as name, size, location, and protection status.
- Directory structures organize files and range from simple single-level tables to hierarchical tree

structures and acyclic graphs that support file sharing.

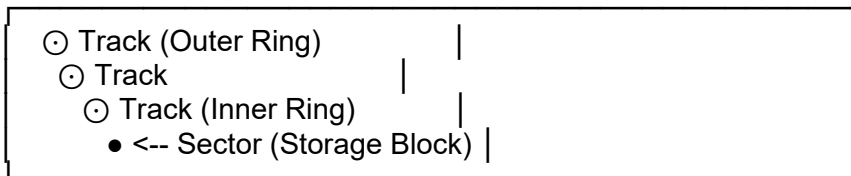
- File allocation methods map logical files to physical disk blocks. Options include contiguous allocation (fast but prone to external fragmentation), linked allocation (flexible but slow for direct access), and indexed allocation (efficiently supports both random and sequential access).

UNIT V — DISK MANAGEMENT

5.1 Disk Structure

- **Physical Architecture:**
 - A hard disk consists of one or more rigid platters. Both surfaces of each platter are coated with a magnetic material used to record data.
 - Each platter surface is divided into concentric rings called **Tracks**.
 - Each track is further subdivided into smaller arcs called **Sectors**, which represent the smallest addressable unit of storage on a disk (typically 512 bytes).
 - A **Cylinder** is the set of all tracks that exist at the same track location across all platter surfaces.
 - A mechanical **Read/Write Head** is positioned over each surface. These heads are attached to a single access arm that moves them in unison across the tracks.

Top-Down View of a Platter Surface:



Performance Metrics:

- **Seek Time:** The time required to move the mechanical disk arm and position the read/write head over the target track.
- **Rotational Latency:** The time it takes for the target sector to rotate under the read/write head.
- **Transfer Time:** The time required to transmit data between the disk medium and the host system.
 $\text{Total Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$
- **Disk Scheduling Goal:** Because seek time is the slowest component of disk access, disk scheduling algorithms aim to minimize total head movement (seek distance) when servicing pending I/O requests.

5.2 Disk Scheduling Algorithms

To compare the following algorithms, we will use a standard sample problem:

- **Disk Range:** 200 tracks (numbered 0 to 199).
- **Pending Request Queue:** 98, 183, 37, 122, 14, 124, 65, 67
- **Initial Read/Write Head Position:** Track 53

5.2.1 FCFS (First Come First Serve)

- **Theory:** Requests are serviced in the exact order they arrive in the I/O queue.

Solved Example — FCFS

- **Execution Path:** 53→98→183→37→122→14→124→65→67
- **Total Seek Calculation:**
 $|98-53|+|183-98|+|37-183|+|122-37|+|14-122|+|124-14|+|65-124|+|67-65|$
 $=45+85+146+85+108+110+59+2=640$ tracks
- **Total Head Movement:** 640 tracks
- **Advantage:** Very simple to implement; fair and free from starvation.
- **Disadvantage:** Results in large seek times due to erratic back-and-forth arm movement.

5.2.2 SSTF (Shortest Seek Time First)

- **Theory:** Selects the request closest to the current head position, minimizing immediate seek distance. This is conceptually similar to Shortest Job First (SJF) scheduling.

Solved Example — SSTF

- **Initial position:** 53
 1. From 53, the closest request is 65 ($|65-53|=12$).
 2. From 65, the closest request is 67 ($|67-65|=2$).
 3. From 67, the closest request is 37 ($|37-67|=30$).
 4. From 37, the closest request is 14 ($|14-37|=23$).
 5. From 14, the closest request is 98 ($|98-14|=84$).
 6. From 98, the closest request is 122 ($|122-98|=24$).
 7. From 122, the closest request is 124 ($|124-122|=2$).
 8. From 124, the closest request is 183 ($|183-124|=59$).
- **Total Seek Calculation:**
 $12+2+30+23+84+24+2+59=236$ tracks
- **Total Head Movement:** 236 tracks
- **Advantage:** Reduces total head movement compared to FCFS.
- **Disadvantage:** Prone to **Starvation**. If new requests continually arrive near the current head position, requests further away may wait indefinitely.

5.2.3 SCAN (Elevator Algorithm)

- **Theory:** The disk arm moves in one direction, servicing all pending requests along the way until it reaches the end of the disk. Once it hits the boundary, it reverses direction and services requests on the way back.

Solved Example — SCAN

(Assume the head is moving toward larger track numbers)

- **Execution Path:** 53→65→67→98→122→124→183→199 (disk boundary)→37→14
- **Total Seek Calculation:**
 $(199-53)+(199-14)=146+185=331$ tracks
- **Note:** The head must travel to the absolute boundary (track 199) before reversing direction.
- **Advantage:** Provides a more uniform wait time under heavy loads while eliminating the risk of starvation.
- **Disadvantage:** Requests at the extreme ends of the disk wait longer than those in the middle because the edges are visited less frequently.

5.2.4 C-SCAN (Circular SCAN)

- **Theory:** Designed to provide more uniform wait times across all tracks. Like SCAN, the head moves in one direction servicing requests until it reaches the end of the disk. However, upon reaching the boundary, it immediately returns to the opposite end (track 0) *without servicing any requests on the return trip*, and then resumes scanning in the original direction.

Solved Example — C-SCAN

(Assume the head is moving toward larger track numbers)

- **Execution Path:** 53→65→67→98→122→124→183→199→0 (fast return jump)→14→37
- **Total Seek Calculation:**
 $(199-53) + (\text{Return jump from 199 to 0}) + (37-0)$
Standard textbook convention counts only active head movement during access:
Active Seek Total = $(199-53) + (37-0) = 146 + 37 = 183$ tracks
- **Advantage:** Provides uniform waiting times across all disk tracks.
- **Disadvantage:** Increases total head movement compared to standard SCAN because of the full return trip across the disk.

5.2.5 LOOK

- **Theory:** A more efficient variation of the SCAN algorithm. The arm moves in one direction servicing requests, but it only travels as far as the *last request* in that direction. It looks ahead to check for pending requests; if none are found, it reverses direction immediately without traveling to the disk boundary.

Solved Example — LOOK

(Assume the head is moving toward larger track numbers)

- **Execution Path:** 53→65→67→98→122→124→183 (highest request; reverse here)→37→14
- **Total Seek Calculation:**
 $(183-53) + (183-14) = 130 + 169 = 299$ tracks
- **Advantage:** Saves seek time by avoiding unnecessary travel to the disk boundaries.

5.2.6 C-LOOK (Circular LOOK)

- **Theory:** A hybrid that combines the circular movement of C-SCAN with the optimization of LOOK. The head moves in one direction servicing requests until it reaches the last pending request. It then performs a fast return jump directly to the *lowest pending request* on the opposite end (rather than track 0) and continues scanning in the original direction.

Solved Example — C-LOOK

(Assume the head is moving toward larger track numbers)

- **Execution Path:** 53→65→67→98→122→124→183→14 (lowest pending request)→37
- **Total Seek Calculation:**
 $(183-53) + (37-14) = 130 + 23 = 153$ tracks
- **Advantage:** One of the most efficient scheduling strategies, minimizing head movement while

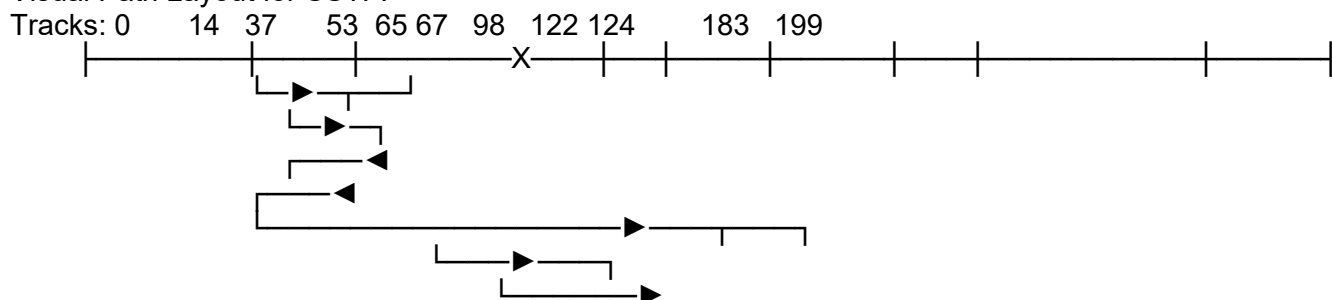
maintaining uniform wait times.

Comparison Table: Disk Scheduling Algorithms

Algorithm	Movement Mechanics	Starvation Risk?	Total Movement (Sample Problem)
FCFS	Sequential arrival order	No	640
SSTF	Nearest request first	Yes	236
SCAN	Bi-directional to disk boundary	No	331
C-SCAN	One direction, full return to track 0	No	183 (excluding return jump)
LOOK	Bi-directional to last request	No	299
C-LOOK	One direction, return to last request	No	153

💡 **Memory Trick:** "Foolish Students Sometimes Cheat, Look Closely" → FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK.

Visual Path Layout for SSTF:



5.3 RAID (Redundant Array of Independent Disks)

- **Definition:** RAID technology combines multiple physical hard drives into a single logical unit. This architecture improves data reliability, system performance, or both, through techniques like striping,

mirroring, and parity.

RAID Levels:

- **RAID 0 (Data Striping):** Data blocks are split evenly across multiple disks.
 - *Fault Tolerance:* None. A single drive failure results in complete data loss.
 - *Benefit:* Maximizes read and write performance.
- **RAID 1 (Disk Mirroring):** Data written to one drive is duplicated onto an identical second drive.
 - *Fault Tolerance:* High. The array can survive the failure of one drive in the pair.
 - *Benefit:* Excellent data redundancy.
- **RAID 5 (Block Striping with Distributed Parity):** Data blocks are striped across drives, and parity data is distributed across all disks in the array.
 - *Fault Tolerance:* High. Can survive the failure of a single drive without data loss.
 - *Benefit:* Balanced performance, capacity, and redundancy.
- **RAID 6 (Block Striping with Double Distributed Parity):** Similar to RAID 5, but features an additional distributed parity block.
 - *Fault Tolerance:* Very High. The array can survive the simultaneous failure of two drives.
- **RAID 10 (RAID 1 + 0):** A hybrid configuration that stripes data across pairs of mirrored drives.
 - *Fault Tolerance:* High. Combines the performance benefits of RAID 0 with the redundancy of RAID 1.

CHAPTER-END QUESTIONS — UNIT V

2 Marks Questions

1. Define Seek Time.
2. What is Rotational Latency?
3. What constitutes a cylinder in disk architecture?
4. Define the operational premise of SSTF scheduling.
5. What is the primary objective of disk scheduling?
6. Differentiate between the execution paths of SCAN and C-SCAN.
7. Define RAID.
8. What is disk striping?

5 Marks Questions

1. Explain FCFS and SSTF disk scheduling using a custom sequence.
2. Explain SCAN and C-SCAN disk scheduling using tracking diagrams.
3. Explain the differences between LOOK and C-LOOK.
4. Explain the structural characteristics of RAID levels 0, 1, and 5.

10 Marks Questions

1. Solve disk scheduling numericals for FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK using a provided track sequence, and draw the corresponding head movement diagrams.

Viva Questions (Unit V)

1. Which disk scheduling algorithm is susceptible to process starvation?
2. What is the structural difference between SCAN and LOOK?
3. Which RAID level uses data mirroring?

4. Which RAID level can tolerate two simultaneous disk drive failures?
5. Why does C-LOOK generally travel fewer tracks than C-SCAN?

Important PYQs (Unit V)

1. Solve disk scheduling numerical problems using SSTF, SCAN, or C-LOOK. (10 marks — Very frequently asked)
2. Explain the characteristics and structures of different RAID levels. (5/10 marks — Common)

Unit V Summary

- Disk access performance is determined by Seek Time, Rotational Latency, and Transfer Time. Disk scheduling algorithms aim to reduce total head movement.
- Algorithms range from basic FCFS to targeted SSTF (closest request first, prone to starvation) and elevator-based algorithms like SCAN, C-SCAN, LOOK, and C-LOOK.
- RAID arrays group physical drives to improve performance and provide fault tolerance. Key configurations include RAID 0 (striping), RAID 1 (mirroring), RAID 5 (distributed parity), and RAID 6 (double distributed parity).

UNIT VI — PROTECTION AND SECURITY

6.1 Introduction

- **Protection:** The internal mechanisms controlled by the OS to govern access of processes and users to system resources.
- **Security:** The broader defensive controls used to safeguard the computer system against external and internal threats (such as malware, hackers, and unauthorized access).
- **Key Distinction:** Protection focuses on internal resource management to ensure processes interact correctly with system assets. Security encompasses protecting the entire system from both internal misuse and external attacks.

6.2 Authentication

- **Definition:** The process of verifying the identity of a user or system entity before granting access.
- **Common Methods:**
 - *Password-based:* The user provides a secret credential (common, but vulnerable if weak passwords are used).
 - *Biometric:* Access is verified using unique physical characteristics (such as fingerprints, retina scans, or facial recognition).
 - *Token-based:* Requires a physical or digital asset, such as a smart card or One-Time Password (OTP).
 - *Multi-Factor Authentication (MFA / 2FA):* Combines two or more distinct verification factors (e.g., a password combined with a temporary mobile token) to enhance access security.

6.3 Authorization

- **Definition:** The process of determining which resources and operations an authenticated user is permitted to access.

💡 **Memory Trick:** Authentication verifies "Who are you?"; Authorization defines "What are you allowed to do?" (Authorization occurs only after successful authentication).

6.4 Access Matrix

- **Definition:** A conceptual model that defines the access rights of subjects (domains, users, or processes) over specific objects (system resources, files, or hardware devices). Rows represent security domains, columns represent objects, and each cell contains the permitted operations.

Sample Access Matrix Layout:

Domain \ Object	File 1	File 2	Printer
D1 (User A)	Read, Write	Read	Execute
D2 (User B)	Read	Read, Write	—
D3 (User C)	—	Execute	Execute

- **Implementation Methods:**
 - **Access Control List (ACL):** The matrix is decomposed by columns. Each object maintains a list of domains along with their specific access privileges.
 - **Capability List:** The matrix is decomposed by rows. Each domain maintains an array of objects along with its access tokens, which act like tickets granting entry.

6.5 Protection Domain

- **Definition:** A protection domain defines the collection of objects a process can access and the specific operations it is allowed to perform on each. It establishes the security boundary for process execution.
- **Domain Switching:** A process can transition from one protection domain to another if permitted by system rules (for example, switching from user mode to kernel mode during a system call).

6.6 Security Threats

6.6.1 Virus

- **Definition:** A malicious program that attaches itself to a legitimate host file or executable. It replicates and spreads when the infected host program is run, often modifying or deleting data.
- **Key Characteristic:** Requires a host program to spread and relies on user action (such as executing an infected file) to activate.

6.6.2 Worm

- **Definition:** A self-contained malicious program that actively replicates and spreads over network

connections without needing to attach to a host file or requiring user intervention.

- **Key Characteristic:** Completely independent; exploits network vulnerabilities to spread automatically, consuming substantial bandwidth and system resources.

6.6.3 Trojan Horse

- **Definition:** A malicious program disguised as legitimate, useful software. Unlike viruses and worms, a Trojan horse does not self-replicate. Instead, it relies on social engineering to trick users into installing it, allowing it to secretly perform harmful actions like stealing data or opening backdoors for unauthorized access.

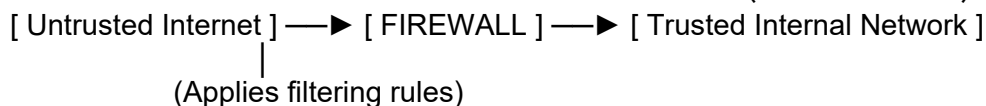
Comparison Table: Malware Threats

Feature	Virus	Worm	Trojan Horse
Requires Host?	Yes	No	No (Masked as legitimate)
Self-Replicating?	Yes (via host)	Yes (independently)	No
Requires User Action?	Yes (running host)	No	Yes (initial installation)
Primary Propagation	File distribution	Network automation	Deception / Social engineering

💡 **Memory Trick:** "A Virus needs a Vehicle (host); a Worm Walks alone; a Trojan Tricks you."

6.7 Firewall

- **Definition:** A hardware- or software-based security system that monitors and filters incoming and outgoing network traffic based on an established set of rules. It acts as a barrier between a trusted internal network and untrusted external networks (like the internet).



- **Types of Firewalls:**
 - *Packet-Filtering:* Inspects individual packets and permits or blocks them based on source/destination IP addresses, ports, and protocols.
 - *Proxy Firewall (Application-Level Gateway):* Acts as an intermediary between systems, inspecting data at the application layer.
 - *Stateful Inspection:* Tracks the state of active network connections to make filtering decisions based on the context of the session.

6.8 Encryption

- **Definition:** The process of converting readable information (plaintext) into an unreadable, encoded format (ciphertext) using a cryptographic algorithm and a key. Authorized parties can decrypt the ciphertext back into plaintext using the correct key.

Plaintext $\xrightarrow{\text{Encryption + Key}}$ Ciphertext $\xrightarrow{\text{Decryption + Key}}$ Plaintext

Types of Encryption:

- **Symmetric Key Encryption:** Uses the same shared key for both encryption and decryption. It is computationally fast but requires a secure method for key distribution.
 - *Examples:* AES, DES.
- **Asymmetric Key Encryption (Public Key):** Uses a mathematically linked key pair: a public key for encryption (which can be shared openly) and a private key for decryption (which must be kept secret). This resolves the key distribution problem but is more computationally intensive.
 - *Examples:* RSA.

Comparison Table: Encryption Implementations

Metric	Symmetric Encryption	Asymmetric Encryption
Keys Required	A single shared key	A public and private key pair
Computational Speed	Fast	Slower
Key Distribution	Difficult (must be kept secret)	Simple (public key can be shared openly)
Standard Algorithms	AES, DES	RSA

CHAPTER-END QUESTIONS — UNIT VI

2 Marks Questions

1. Define Authentication.
2. Define Authorization.
3. What is an Access Matrix?
4. Differentiate between a virus and a worm.
5. What is a Firewall?
6. Define Encryption.
7. What is a Trojan Horse?
8. What constitutes a Protection Domain?

5 Marks Questions

1. Explain the Access Matrix model using a practical example.
2. Explain the differences between a Virus, a Worm, and a Trojan Horse.
3. Explain the types of firewalls used to secure networks.
4. Explain Symmetric and Asymmetric encryption techniques with examples.

10 Marks Questions

1. Explain Authentication and Authorization mechanisms, including multi-factor and biometric systems.
2. Explain the Access Matrix model and its implementation strategies (Access Control Lists and Capability Lists).
3. Provide a comprehensive analysis of security threats (including Viruses, Worms, Trojan Horses, Logic Bombs, and Ransomware) and explain how firewalls and encryption help protect systems.

Viva Questions (Unit VI)

1. What is the core difference between protection and security?
2. What does 2FA stand for?
3. Which encryption method is faster: symmetric or asymmetric?
4. What is a capability list?
5. Can a worm propagate without user intervention?

Important PYQs (Unit VI)

1. Explain the Access Matrix model and its implementations. *(5/10 marks — Frequently asked)*
2. Differentiate between a Virus, a Worm, and a Trojan Horse. *(5 marks — Very common)*
3. Explain Encryption types and give examples of their use. *(5/10 marks)*

Unit VI Summary

- Authentication verifies identity, while Authorization defines access permissions.
- The Access Matrix model maps domain privileges over system objects. It can be implemented using column-based Access Control Lists (ACLs) or row-based Capability Lists.
- Malware threats include viruses (which require a host file), worms (which spread automatically across networks), and Trojan horses (which rely on deception).
- Firewalls filter traffic between networks, while encryption protects data confidentiality using either symmetric (single shared key) or asymmetric (public/private key pair) algorithms.

MASTER QUESTION BANK & FINAL REVISION SECTION

A. ONE MARK / MCQ QUESTIONS (SAMPLE SET — UNIT-WISE)

Unit I

1. An operating system acts as an interface between the **user** and the **computer hardware**.
2. **Time-sharing** scheduling provides the illusion of a dedicated CPU to multiple users.

3. PCB stands for **Process Control Block**.
4. A program in active execution is known as a **process**.
5. The state where a process is waiting to be assigned to the CPU is the **ready state**.
6. RTOS stands for **Real-Time Operating System**.
7. In a Distributed OS, the network of computers appears to the user as a **single system**.
8. An OS that utilizes multiple CPUs within a single system is a **multiprocessing system**.
9. Saving the state of an active process and restoring the state of another is called **context switching**.
10. Hard real-time systems **do not** tolerate missed deadlines.

Unit II

11. FCFS stands for **First Come First Serve**.
12. **SRTF** scheduling yields the absolute minimum average waiting time for a set of processes.
13. Waiting Time = Turnaround Time minus **Burst Time**.
14. In Round Robin scheduling, each process is allocated a fixed time slice called a **time quantum**.
15. Deadlock requires **four** necessary conditions to occur simultaneously.
16. The Banker's Algorithm was originally proposed by **Edsger Dijkstra**.
17. RAG stands for **Resource Allocation Graph**.
18. A cycle in a RAG where resources have single instances always indicates a **deadlock**.
19. Shortest Job First scheduling can cause **starvation** for long-running jobs.
20. The technique used to prevent starvation in priority scheduling is called **aging**.

Unit III

21. Paging divides physical memory into fixed-size blocks called **frames**.
22. Segmentation reflects the **logical / user** view of memory.
23. Allocating fixed-size blocks can lead to **internal** fragmentation.
24. TLB stands for **Translation Lookaside Buffer**.
25. The **FIFO** page replacement algorithm is susceptible to Belady's Anomaly.
26. Thrashing occurs when a system spends too much time handling **page faults**.
27. Optimal page replacement requires advance **future** knowledge of references.
28. Demand paging loads pages into memory only when they are **referenced / demanded**.
29. Temporarily moving an entire process out of memory to disk is called **swapping**.
30. Physical Address = (Frame Number × Page Size) + **Offset**.

Unit IV

31. The smallest logical unit of storage managed by an OS is a **file**.
32. FAT stands for **File Allocation Table**.
33. **Linked** allocation eliminates external fragmentation but only supports sequential access.
34. UNIX systems implement indexed allocation using data structures called **inodes**.
35. The **general graph** directory structure allows cycles to exist.
36. In a UNIX permission string, rwx indicates full **read, write, and execute** permissions.

Unit V

37. The time required to position the disk arm over the target track is called **seek time**.
38. The **SSTF** algorithm services the pending request closest to the current head position first.
39. C-LOOK differs from LOOK because it **returns directly to the lowest requested track** rather than reversing direction sequentially.
40. RAID level 1 provides redundancy through data **mirroring**.

41. RAID level 5 balances performance and fault tolerance using distributed **parity**.
42. RAID level 0 provides performance benefits but **no** fault tolerance.

Unit VI

43. Verifying the identity of a user or system entity is called **authentication**.
44. Granting specific access permissions to a user is called **authorization**.
45. A **virus** requires a host program or file to propagate.
46. A **worm** can replicate and spread across networks independently.
47. A **Trojan horse** is malicious software disguised as a legitimate application.
48. AES is an example of a **symmetric** encryption algorithm.
49. RSA is an example of an **asymmetric** encryption algorithm.
50. A firewall filters traffic passing through a **network connection**.

B. TWO MARK QUESTIONS — CONSOLIDATED LIST

1. Define an Operating System and name two examples.
2. What is the core difference between a process and a program?
3. What is a PCB? List four fields it contains.
4. Define context switching.
5. What is a time-sharing operating system?
6. Define multiprocessing and distinguish between SMP and AMP.
7. What is a Real-Time Operating System?
8. Define CPU Scheduling.
9. State the algebraic formulas for calculating Turnaround Time and Waiting Time.
10. What is the Convoy Effect?
11. Define Deadlock.
12. List the four necessary conditions for a deadlock to occur.
13. What defines a safe state in deadlock avoidance?
14. Define starvation and explain how aging resolves it.
15. What is a Resource Allocation Graph?
16. Differentiate between preemptive and non-preemptive scheduling.
17. Define fragmentation and distinguish between its internal and external forms.
18. What is Paging?
19. What is Segmentation?
20. Define virtual memory.
21. What is demand paging?
22. Define thrashing.
23. What is swapping?
24. What is Belady's Anomaly?
25. Define the function of a Translation Lookaside Buffer (TLB).
26. What is a file?
27. List four common file attributes.
28. What is a directory?
29. Define contiguous file allocation.
30. What is a File Allocation Table (FAT)?
31. Differentiate between sequential access and direct access.
32. Define seek time and rotational latency.
33. Explain Shortest Seek Time First (SSTF) scheduling.
34. Differentiate between the SCAN and C-SCAN disk scheduling algorithms.
35. Define RAID and state its primary engineering purposes.

36. Distinguish between authentication and authorization.
37. What is an Access Matrix?
38. Differentiate between a virus and a worm.
39. What is a firewall?
40. Define encryption and state its two main implementations.

C. FIVE MARK & LONG (10 MARK) COMPREHENSIVE INDEX

Review the designated essay questions listed at the end of Units I through VI. These cover the following core topics:

- **Unit I:** OS Classifications, Process States, and PCB Structure.
- **Unit II:** CPU Scheduling Algorithms (with numerical charts), Deadlock Conditions, and the Banker's Algorithm.
- **Unit III:** Paging, Segmentation, Page Fault Handling, and Page Replacement Algorithms (FIFO/LRU numerical matrices).
- **Unit IV:** File Allocation Methods and Directory Structures.
- **Unit V:** Disk Scheduling Algorithms (Path Diagrams) and RAID Level Configurations.
- **Unit VI:** Access Control Models, Malware Threats, Firewalls, and Encryption Implementations.

 **Exam Strategy Tip:** BCA Operating System exams typically follow a predictable format: a set of short 2-mark definitions, a series of 5-mark conceptual explanations, and a few long-form 10-mark essay or numerical problems. Numerical problems (such as CPU scheduling charts, the Banker's safe sequence test, page replacement tables, and disk scheduling calculations) carry significant weight and are excellent opportunities to score full marks if practiced carefully.

D. CONSOLIDATED VIVA-VOCE QUESTIONS

1. **What is the core function of an OS?** It acts as an intermediary interface between user software and the underlying physical hardware, managing system resources efficiently.
2. **What is the difference between a process and a thread?** A process is an independent program in execution with its own address space. A thread is a lightweight unit of execution that belongs to a process and shares its address space and resources.
3. **What is the ready queue?** A memory queue maintained by the OS that holds all processes that are ready to execute and waiting for CPU time.
4. **Explain the five process states.** New (being created), Ready (waiting for CPU), Running (executing instructions), Waiting (blocked, waiting for an event or I/O), and Terminated (finished execution).
5. **What information does a PCB hold?** It stores execution tracking data for a process, including the PID, process state, program counter, CPU registers, memory limits, and open file lists.
6. **What is context switching and why is it considered overhead?** It is the process of saving the state of a running process and loading the state of another. It is considered overhead because the CPU performs no useful application work while executing the switch.
7. **Which CPU scheduling algorithm yields the lowest average waiting time?** Shortest Remaining Time First (SRTF) for preemptive scheduling, and Shortest Job First (SJF) for non-preemptive scheduling.
8. **What is starvation and how is it resolved?** Starvation occurs when a process waits indefinitely for a resource or the CPU because other processes are prioritized. It is resolved using **aging**, which gradually increases the priority of a process as it waits.
9. **What are the four Coffman conditions for a deadlock?** Mutual Exclusion, Hold and Wait, No

Preemption, and Circular Wait.

10. **Explain the Banker's Algorithm in one line.** It is a deadlock avoidance algorithm that dynamically evaluates resource requests and grants them only if the resulting system state remains safe.
11. **What is a safe sequence?** An ordering of processes that guarantees every process can acquire its maximum resource demands and run to completion without causing a deadlock.
12. **What is the difference between deadlock prevention and deadlock avoidance?** Deadlock prevention uses structural rules to ensure at least one of the four necessary conditions cannot hold. Deadlock avoidance uses dynamic analysis to ensure the system never enters an unsafe state.
13. **What is the difference between internal and external fragmentation?** Internal fragmentation occurs when allocated memory is slightly larger than requested, wasting space *inside* the allocated block. External fragmentation occurs when free memory is broken into small, non-contiguous holes scattered across the system, preventing it from being allocated to a new process.
14. **What is a page fault?** An interrupt triggered by the memory management unit when a process attempts to access a page that is not currently loaded into physical RAM.
15. **Why is a TLB used?** A Translation Lookaside Buffer (TLB) acts as a fast hardware cache that stores recent logical-to-physical address translations, reducing memory access latency.
16. **What causes thrashing and how is it resolved?** Thrashing is caused by an excessively high degree of multiprogramming, where processes compete for too few frames and spend more time paging than executing. It is resolved by reducing the number of active processes or allocating more frames using the Working Set Model.
17. **Differentiate between paging and segmentation.** Paging divides memory into fixed-size blocks (pages and frames) from a physical perspective. Segmentation divides memory into variable-sized blocks based on the logical, programmer's view of the application.
18. **Which page replacement algorithm is optimal but impractical?** The Optimal Page Replacement algorithm (OPT), which replaces the page that will not be used for the longest duration in the future. It is impractical because it requires advance knowledge of the future reference string.
19. **What is Belady's Anomaly?** A phenomenon where increasing the number of physical frames in memory results in an *increase* in the number of page faults. This occurs in the FIFO page replacement algorithm.
20. **What is the difference between linked and indexed file allocation?** Linked allocation connects scattered disk blocks using pointers embedded within each block. Indexed allocation collects all block pointers into a single centralized index block, allowing direct access.
21. **Name the five common directory structures.** Single-level, Two-level, Tree-structured, Acyclic Graph, and General Graph directories.
22. **What is the difference between seek time and rotational latency?** Seek time is the time required to move the mechanical disk arm to the correct track. Rotational latency is the time it takes for the target sector to rotate under the read/write head.
23. **Which disk scheduling algorithm operates like an elevator?** The SCAN algorithm.
24. **What is the difference between the SCAN and LOOK algorithms?** SCAN travels all the way to the absolute disk boundary before reversing direction. LOOK only travels as far as the last pending request in that direction before reversing.
25. **Name two RAID configurations that provide fault tolerance.** RAID 1 (mirroring) and RAID 5 (distributed parity).
26. **What is the difference between authentication and authorization?** Authentication verifies the identity of a user ("Who are you?"). Authorization determines the access permissions and actions allowed for that user ("What can you do?").
27. **What is an Access Matrix?** A security model used to define the access rights of different domains or users over specific system objects and resources.
28. **Differentiate between a virus, a worm, and a Trojan horse.** A virus attaches itself to a host file and requires user action to spread. A worm is a standalone program that replicates and spreads across networks automatically. A Trojan horse disguises itself as legitimate software but performs

harmful actions secretly.

29. **What does a firewall do?** It monitors and filters network traffic between a trusted internal network and an untrusted external network based on an established set of security rules.
30. **Differentiate between symmetric and asymmetric encryption.** Symmetric encryption uses a single shared key for both encryption and decryption. Asymmetric encryption uses a mathematically linked key pair: a public key for encryption and a private key for decryption.

E. QUICK REFERENCE FORMULA SHEET

CPU Scheduling:

- Turnaround Time (TAT)=Completion Time (CT)–Arrival Time (AT)
- Waiting Time (WT)=Turnaround Time (TAT)–Burst Time (BT)
- Response Time (RT)=Timestamp of first CPU allocation–Arrival Time (AT)
- Average Waiting Time= $\frac{\text{Total number of processes} \times (\sum \text{Waiting Time of all processes})}{\text{Total number of processes}}$

Deadlock (Banker's Algorithm):

- $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$
- $\text{Available} = \text{Total System Resources} - \sum \text{Allocated Resources}$
- *Safety Condition Check:* If $\text{Need}[i] \leq \text{Work} \rightarrow \text{Grant allocation and update: } \text{Work} = \text{Work} + \text{Allocation}[i]$

Memory Management (Paging):

- $\text{Page Number} = \lfloor \frac{\text{Page Size Logical Address}}{\text{Page Size}} \rfloor$ (Integer division block result)
- $\text{Offset} = \text{Logical Address} \pmod{\text{Page Size}}$ (Remainder block result)
- $\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$

Disk Scheduling:

- $\text{Total Head Movement} = \sum |\text{Current Track Number} - \text{Next Scheduled Track Number}|$
- $\text{Total Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$

F. PRINCIPAL ARCHITECTURAL COMPARISONS

- **CPU Scheduling Algorithms:**
 - *FCFS:* Simple to implement but prone to the convoy effect.
 - *SJF:* Minimizes waiting times for non-preemptive setups but can cause starvation for long jobs.
 - *SRTF:* Preemptive, optimal choice for minimal waiting times, but incurs high scheduling overhead.
 - *Priority:* Prioritizes critical workflows but requires aging to prevent starvation.
 - *Round Robin:* Equitable time-sharing approach, but performance depends heavily on the selected time quantum.
- **Deadlock Handling Strategies:**
 - *Prevention:* Imposes static structural rules to prevent deadlocks, resulting in low resource utilization.
 - *Avoidance:* Dynamically evaluates resource requests using algorithms like the Banker's safety check to maintain a safe state.
 - *Detection & Recovery:* Allows deadlocks to occur, periodically detects them, and recovers by terminating processes or preempting resources.

- **Page Replacement Options:**
 - *FIFO*: Simple queue-based replacement, but prone to Belady's Anomaly.
 - *LRU*: Practical approach based on recent usage history; provides stable performance but requires hardware support.
 - *Optimal*: Theoretical best with minimal page faults; used primarily as a baseline since it requires future knowledge.
- **File Allocation Approaches:**
 - *Contiguous*: High access speeds for random and sequential access, but prone to external fragmentation.
 - *Linked*: Flexible and avoids fragmentation, but limited to sequential access and prone to pointer corruption issues.
 - *Indexed*: Balanced approach that supports fast direct access and dynamic file growth; standard model for UNIX inodes.
- **Disk Scheduling Strategies:**
 - *FCFS*: Simple and fair, but results in high seek times.
 - *SSTF*: Minimizes immediate seek distances but carries a risk of starvation.
 - *SCAN/LOOK*: Elevator-style algorithms that move across tracks, reducing head movement and preventing starvation.
 - *C-SCAN/C-LOOK*: Circular scanning models that provide uniform wait times across all tracks.
- **Security Malware Types:**
 - *Virus*: Dependent on a host file and requires user action to execute and spread.
 - *Worm*: Independent program that propagates automatically across networks by exploiting vulnerabilities.
 - *Trojan Horse*: Relies on deception to trick users into installing it, then secretly compromises the system.

G. LAST NIGHT REVISION NOTES (CRAM SHEET)

- **OS Core Function:** An interface between the user and hardware. It manages the CPU, memory, files, devices, security, and user interfaces.
- **OS Classifications:** Batch, Multiprogramming, Multitasking, Multiprocessing (SMP vs. AMP), Time-Sharing, RTOS (Hard vs. Soft constraints), Distributed, and Network systems.
- **Process Execution States:** New → Ready → Running → Waiting → Terminated.
- **PCB Components:** Stores the PID, process state, program counter, CPU registers, memory limits, and I/O allocation data.
- **Scheduling Calculations:** $TAT = CT - AT$, and $WT = TAT - BT$.
- **Scheduling Performance:** SJF provides optimal waiting times for non-preemptive systems, SRTF is optimal for preemptive systems, and Round Robin provides fair allocation for time-sharing systems.
- **Deadlock Conditions:** All four Coffman conditions must hold simultaneously: Mutual Exclusion, Hold & Wait, No Preemption, and Circular Wait.
- **Banker's Avoidance Matrix:** Calculate $Need = Max - Allocation$. Grant resource allocations only if $Need \leq Work$, ensuring the system remains in a safe sequence.
- **Fragmentation Types:** Internal fragmentation occurs within fixed allocation blocks (common in paging). External fragmentation consists of scattered, non-contiguous free blocks (common in segmentation and contiguous allocation) and can be resolved using compaction.
- **Paging vs. Segmentation:** Paging divides memory into fixed-size blocks from a physical system perspective. Segmentation organizes memory into variable-sized blocks based on the logical, user perspective.
- **Page Replacement Algorithms:** FIFO is simple but susceptible to Belady's Anomaly. LRU is a practical and effective algorithm based on past usage history. Optimal provides the lowest page fault rate but is a theoretical model only.

- **Thrashing:** Occurs when a system spends more time swapping pages than executing instructions. It is resolved by allocating adequate frames via the Working Set Model or reducing the number of active processes.
- **File Allocation Approaches:** Contiguous (fast but prone to fragmentation), Linked (flexible but restricted to sequential access), and Indexed (balanced, supports direct access, standard for UNIX).
- **Directory Structures:** Progress from simple single-level lists to structured hierarchical trees and flexible acyclic graphs that support file sharing.
- **Disk Access Performance:** Minimizing seek time is the primary goal. C-LOOK and C-SCAN are highly efficient algorithms that provide uniform wait times across tracks.
- **RAID Levels:** RAID 0 provides performance through striping but no safety. RAID 1 provides redundancy through mirroring. RAID 5 distributes parity across drives, and RAID 6 features double distributed parity.
- **System Security:** Authentication verifies identity, while Authorization checks permissions. Viruses require host files to spread, worms propagate automatically across networks, and Trojans rely on deception.
- **Data Encryption:** Symmetric encryption uses a single shared key and is fast (e.g., AES). Asymmetric encryption uses a public/private key pair, resolving the key distribution problem (e.g., RSA).

H. PYQ MATRIX ANALYSIS & EXPECTED SYLLABUS WEIGHTAGE

Unit	Expected Weightage	Primary Question Types & Repeated Topics
I	15%	OS Classifications, Process State Transitions, PCB Structure and Fields.
II	25%	CPU Scheduling numerical calculations (Gantt charts), Banker's Algorithm safety matrix problems. <i>Highest priority unit.</i>
III	20%	Paging and Segmentation comparisons, Page Fault handling, Page Replacement numerical calculations (FIFO/LRU tables).
IV	15%	File Allocation methods (Contiguous, Linked, Indexed), Hierarchical and

Unit	Expected Weightage	Primary Question Types & Repeated Topics
		Graph Directory structures.
V	15%	Disk Scheduling numerical path calculations (SSTF, SCAN, C-LOOK), RAID level comparisons.
VI	10%	Access Matrix configurations (ACL vs. Capability lists), Malware threat classifications, Symmetric vs. Asymmetric encryption.

 **The Golden Rule for OS Success:** Units II, III, and V contain the numerical tracking problems. These questions carry significant weight and are straightforward opportunities to earn full marks if you practice them carefully. Work through every solved example in this handbook by hand using pen and paper before entering the exam hall.

FINAL EXAM-DAY ARCHITECTURAL TIPS

1. **Draw Gantt Charts and Diagrams Clearly:** Always include structural diagrams and Gantt charts for scheduling, deadlock, memory, and disk path questions. Evaluators allocate explicit points for accurate visual diagrams.
2. **State Formulas Explicitly:** Write out the general formula before plugging in numerical values. If you make a minor arithmetic error, showing the correct formula can still earn you partial credit.
3. **Structure Conceptual Explanations:** For long-form questions (e.g., "*Explain X algorithm with an example*"), organize your answer systematically: **Definition → Operational Diagram → Steps / Pseudocode → Worked Numerical Example → Advantages → Disadvantages.**
4. **Use Formatting to Improve Readability:** Keep your answers concise and point-wise for 2-mark and 5-mark questions. For 10-mark questions, use clear headings and structured paragraphs.
5. **Review the Cram Sheet:** Read through the *Last Night Revision Notes* right before the exam to keep key terms, definitions, and formulas fresh in your mind.

END OF HANDBOOK
